Engineering Applications of Artificial Intelligence 100 (2021) 104196

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

# Two efficient nature inspired meta-heuristics solving blocking hybrid flow shop manufacturing problem

Said Aqil *, Karam Allali

*Laboratory Mathematics, Computer Science and Applications, University Hassan II of Casablanca, FST, PO Box 146, Mohammedia, Morocco*

ARTICLE INFO

*Keywords:*
Hybrid flow shop
Water wave optimization algorithm
Migratory bird optimization algorithm
Blocking
Total tardiness and earliness

ABSTRACT

The hybrid flow shop scheduling problem is one of the most relevant optimization problem in manufacturing industry. In this paper, we investigate the blocking hybrid flow shop scheduling problem under the constraint of sequence dependent setup time. The objective is to minimize the total tardiness and earliness with uniform parallel machines under the constraint of sequence dependent setup time. To solve this kind of problems, significant developments of new meta-heuristic algorithms make it possible to implement new metaheuristics inspired by the behavior of living beings or natural phenomena. In this context, we suggest six algorithms based on the migratory bird optimization and the water wave optimization algorithms. We give three new versions for each meta-heuristic in order to solve this optimization problem. The main improvement of the suggested algorithms concerns the exploration phase of the neighborhood system. The enhancement approaches are based on the iterated greedy algorithm, the greedy randomized adaptive search procedure, the path relinking technique and the local search procedures. These modifications in the two nature inspired meta-heuristics make it possible to develop a new neighborhood generation structure constituting hybrid optimization algorithms. A comparative study between the different proposed methods is carried out on a variety of problems ranging from small to relatively large size instances. The simulations show good performances recorded by the water wave optimization algorithm in term of quality and convergence speed towards the best solution.

## 1. Introduction

The blocking hybrid flow shop scheduling (BHFS) problem is one of the most important optimization problems in industrial manufacturing. Indeed, this issue consists of two sub-problems: the first is the blocking flow shop scheduling problem (BFSP) and the second is the parallel machines scheduling problem. The situation of blocking constraint is usually met when the storage space is insufficient for finished jobs or for jobs in progress. The blocking job scheduling problem can be encountered in the manufacturing industry. The interest of the blocking is to keep the job position in the machine to respect the storage capacity or to not lose some of the job qualities. This case is frequently encountered in food production, metallurgy industry, continuous petrochemical production, etc. Much research has focused on the field of the hybrid flow shop scheduling (HFS) optimization problem.

Currently, many companies are forced to install the machines in the workshops in small spaces. This situation often leads to the constraint of blocking jobs in machines or in handling systems. The goal is to efficiently manage the flow processes and to guarantee the production of jobs without losing the trust of their customers. In this paper, we are interested in solving blocking hybrid flow shop scheduling problem. The objective is to minimize the total tardiness and earliness with uniform parallel machines (UPM) under constraints of blocking and of sequence dependent setup time (SDST). Indeed, these two constraints have a relevant interest in several optimization scheduling case studies.

We note that recent studies on the BHSF problem aim at optimizing one or several scheduling criteria. In Mollaei et al. (2019), a model solving BHFS with mixed linear integer programming (MILP) method is proposed to solve makespan and total production cost bi-objective optimization issue. Also, Zhang (2019) develops MILP model and a genetic algorithm (GA) to solve the BHFS problem under SDST constraint, whose goal is to minimize the makespan. In Moccellin et al. (2018), a set of heuristics is proposed to solve the BHFS problem with various cases of the setup time configurations. Recently, in Lin et al. (2020) a hybrid harmony search and GA have been proposed to solve reentrant HFS problem with limited buffer capacity and stokers by minimizing the makespan and the mean flow time. Motivated by these previous works, we propose an optimization model to minimize the total tardiness and earliness for the BHFS-SDST problem. In addition to the blocking and SDST constraints, we will assume that each job has a realize and the due dates. The resources forming each HFS manufacturing stage are constituted by several uniform parallel machines. The particularity of the studied problem and the nature of the imposed

* Corresponding author.
*E-mail addresses:* s_aqil@hotmail.fr (S. Aqil), allali@hotmail.com (K. Allali).

https://doi.org/10.1016/j.engappai.2021.104196
Received 10 March 2020; Received in revised form 13 December 2020; Accepted 6 February 2021
Available online 20 February 2021
0952-1976/© 2021 Elsevier Ltd. All rights reserved.

constraints present a strong and attractive interest in the manufacturing industry. Hence, our model reflects a real scheduling problem with realistic constraints concerning the machines, taking into account the earliest and due dates of jobs. Indeed, in many manufacturing processes, the jobs do not have the same release date. This constraint appears when the company gathers several orders from its customers. The goal is to exploit in the best way the production resources for a large batch. The constraint of deadlines imposes job delivery at the best time with respect to the due dates; otherwise, they will be considered late. Thus, our model brings together constraints relating to the management of the machine park and the satisfaction of the customers. The model will be noted $HFS((QM)_l)_{l=1}^{L}|r_j, d_j, SDST, blocking|\sum_{j=1}^{n}(E_j + T_j)$ respecting the notation adopted in the scheduling literature. We note that this notation contains three fields $\alpha|\beta|\gamma$ adopted by Graham et al. (1979). This notation is improved by Vignier et al. (1999). From the state of the art and classification of HFS problems presented by Ruiz and Vázquez-Rodríguez (2010) and Ribas et al. (2010), we can notice that there is little work dealing with the presence of UPM in HFS scheduling problems.

Our contribution is to solve a relatively specific HFS scheduling optimization problem. The proposed model combines the constraints of earliest dates, deadlines as well as the blocking of jobs and finally the setup times of machines. The goal is to minimize total tardiness and earliness on HFS problem subject to blocking and SDST constraints. The relevance and the particularity of studying UPM problem is highlighted in Li et al. (2019, 2018). Besides, the BFSP scheduling model under constraint SDST presents one of the most important subjects in the modern manufacturing process (Aqil and Allali, 2019; Newton et al., 2019; Takano and Nagano, 2019). Our model, therefore, combines the two previous sub-models in a single study constituting the BHFS-SDST model. We suggest new versions of solving algorithms based on two nature inspired metaheuristics. The first one is based on the migratory birds optimization (MBO) algorithm and the second one is based on the water wave optimization (WWO) algorithm. In addition, we propose a variety of enhancement procedures in the local search phase for each algorithm. We propose the iterated greedy (IG) procedure, the greedy randomized adaptive search procedure (GRASP) as well as the iterative local search (ILS) algorithms. We will apply our meta-heuristics in multiple versions to solve the problem, in total of six algorithms, three for MBO and three others for WWO. Our scheduling problem is inspired by a real industrial case highlighting UPM on each stage with the presence of constraints of blocking and SDST. Indeed, the manufacturing application system that we will choose in our study is the painting shop production process. The ordered items will be sorted into several categories inducing specific qualities and colors. The workshop processes are composed of four main phases: the crushing of natural stones, the shear mixing with height speed, the grinding phase and finally the mixture phase.

Fig. 1 shows the production process in the painting shop manufacturing problem. This workflow system is equipped at each stage with several UPM with proportional performances. Each job will be processed by a single machine at each phase; special tools are mounted in the machine executing the processed job. The tools are assembled and dis-assembled in order that the new job must be processed. This leads to additional time in the manufacturing process. In the last three phases, jobs are in the form of a liquid mixture that will be poured by filling the machine equipment tanks. The jobs have different colors requiring time to lift the tanks when each job is completed on the machine. The machines in this manufacturing are characterized by cutting tools with removable blades; the maintenance as well as the adjustment of the tools will be considered as SDST time. Each operator is specialized in a machine, and each machine has a different used technology. We note that the transit of jobs between the production phase is ensured by trolleys driven by skilled workers. The launched jobs in the production cycle come from different orders from the customers, which implies a realize date of each product. The workshop is modeled as a hybrid flow shop under constraints of setup time and arrival time of jobs. The objective is to meet the expected dates for the final delivery of the batch, minimizing the total tardiness and earliness. We take inspiration from the paint manufacturing workshop model to generalize it for several stages. Our approach to solve the HFS problem is, therefore, to propose a resolution model based on two inspired nature metaheuristics. Both metaheuristics are implemented in various versions that are subject to improvement in the neighborhood exploration phase.

**Table 1**
Processing time of five jobs in three machines.

| Job | $J_1$ | $J_2$ | $J_3$ | $J_4$ | $J_5$ |
|---|---|---|---|---|---|
| $M_1$ | 8 | 6 | 8 | 6 | 10 |
| $M_2$ | 12 | 10 | 8 | 8 | 8 |
| $M_3$ | 10 | 12 | 8 | 8 | 6 |

In our study, we propose a set of resolution approaches for the BHFS-SDST problem. This problem is composed of two main sub-problems, that of the BFSP and that of the HFS. We illustrate the two cases of scheduling problems using a numerical example. For this, we consider a set of five jobs $J = \{1, 2, 3, 4, 5\}$ and three machines $M_1, M_2, M_3$ forming three stages for the BFSP problem. In Table 1 we give the processing times of each job on each machine. The launched sequence at the start of the production cycle is $\pi = [2, 1, 3, 5, 4]$.

We represent the Gantt chart of the scheduling in Fig. 2 on the three machines. To simplify the study, we consider minimizing the maximum completion time of scheduling called the makespan $C_{max}$. From this representation, we see that the value of $C_{max}$ is 62 units of the time given by the end date of the last job in the sequence on the machine $M_3$. We can also notice that the jobs $1, 3, 4$ will be blocked in machine $M_1$ with respective time blocking are 2,4 and 2 units of time. In addition, only the job 3 will be blocked in the machine $M_2$ with 2 time units.

To illustrate the scheduling in a three-stage HFS workshop, we are studying the same example made up of five jobs. However, we consider that the batch of jobs will be executed on three stages with two identical parallel machines on each stage. The machines on stage 1 are $M_{11}, M_{21}$, those on stage 2 are $M_{12}, M_{22}$ and finally the machines on stage 3 are $M_{13}, M_{23}$. The scheduled sequence at the start of the scheduling is $\pi = [2, 1, 3, 5, 4]$, each job will be executed on the available machine as soon as possible. Each job will only be processed by one machine, and each machine will only run one job at a time. At the end of each stage, all jobs will be sorted in ascending order of their end dates to be processed by machines on the next stage.

In the Fig. 3, we give the representative Gantt chart of the evolution of the execution of the batch of five jobs. The value of the makespan in this sequence is 44 time units given by the end date of the job 4 on the machine $M_{22}$ on the stage 3. We note that the job 5 is finished before the job 3 in the final stage, this means that it will swap its position with the job 3 in the exit sequence.

In our work, we mentioned that the BHFS-SDST scheduling problem is the general case of the BFSP-SDST problem. Indeed, starting from a HFS problem with a parallel machine per stage under SDST constraints, we obtain the HFS-SDST problem. When the blocking constraint is added to the latter problem, the problem is named BHFS-SDST. However, from the HFS problem, when there is only one machine per stage with the SDST constraint, the problem is noted PFSP-SDST. When the blocking constraint is respected in the latter, the problem will be identified as a BFSP-SDST problem. Finally, we move from the BHFS-SDST problem to the BFSP-SDST problem by reducing the number of machines per stage to a single machine.

In the Fig. 4, we describe the connection between the BHFS-SDST and BFSP-SDST scheduling problems. We also specify the constraints of passage between the different problems described in this diagram. We note that when the SDST constraint is not considered for the problem BFSP and it will be taken in the problem BHFS, one can establish

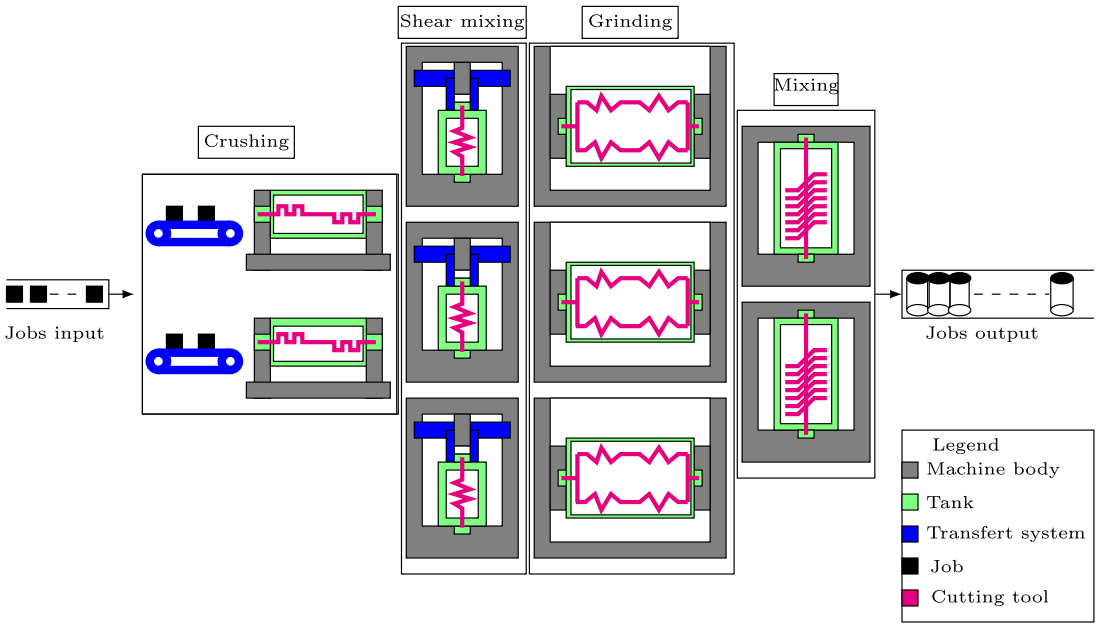



**Fig. 1.** Paint manufacturing processes.

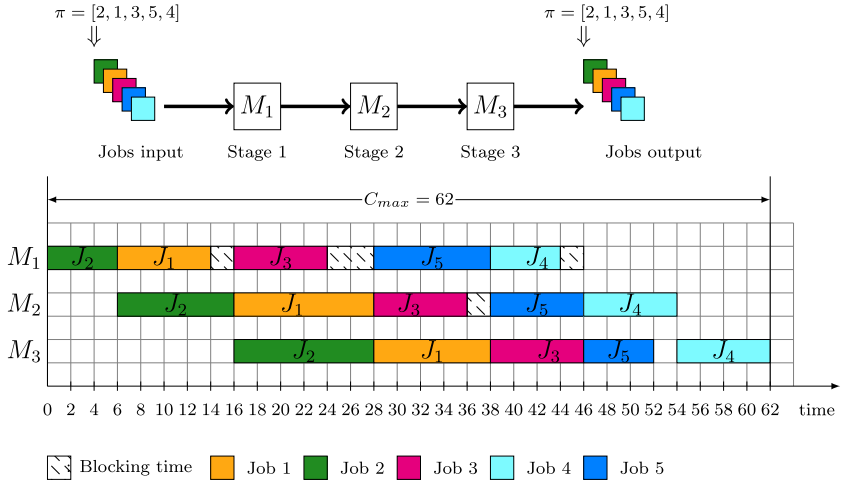



**Fig. 2.** Gantt chart for BFSP problem with three machines.

the diagram of connection between the problem BHFS-SDST and the problem BFSP. Our study deals with minimizing total tardiness and earliness BHFS-SDST under constraints of realize and due dates.

Our paper will be organized according to the following description. In Section 2, we present a review of past works, followed in Section 3 by a description of the studied model. In Section 4, we present the proposed solving approaches. In Section 5, a simulation study is described and at the end of the paper, Section 6 will conclude this contribution by focusing on the strengths of our approaches as well as the perspectives for future works.

## 2. Related works

The strong presence of the HFS scheduling problem in the production or service industry has been the subject of several research studies. The developed approaches present theoretical and practical models for solving the studied problems. The developed approaches are often based on computer simulation models. Consequently, the proposed models constitute a decision-making aid tool for business managers to ensure efficient management of the machine park. We propose a review of literature on solving the HFS problem based on various approaches of resolution. We focus on two main methods: exact methods and approximate methods based on heuristics and metaheuristics. We note that exact methods are based on mixed integer linear programming (MILP) or branch and bound (BB) methods. The last method depends on lower bounds (LB) or upper bounds (UB). We indicate that the development of exact methods is often employed to solve the problems relatively of small size instances. Besides, approximate methods are often used to solve the scheduling problem with relatively large size instances. Over the last two decades, these methods have been the subject of several research studies on solving HFS problem. The work (Jun and Park, 2015) developed an implementation of the GA to solve the HFS problem. Subsequently, another approach based on GA is applied to solve HFS re-entrant manufacturing problem (Chamnanlor et al., 2017). Also, Kianfar et al. (2012) developed the GA using a set of heuristics to solve the stochastic HFS problem in order to minimize the average tardiness of jobs. Likewise, in Yu et al. (2018) GA is proposed to solve the HFS problem with unrelated parallel machines under the constraint of machine eligibility. In Cui and Gu (2015), an improved version of the artificial bee colony (ABC) algorithm is suggested to minimize

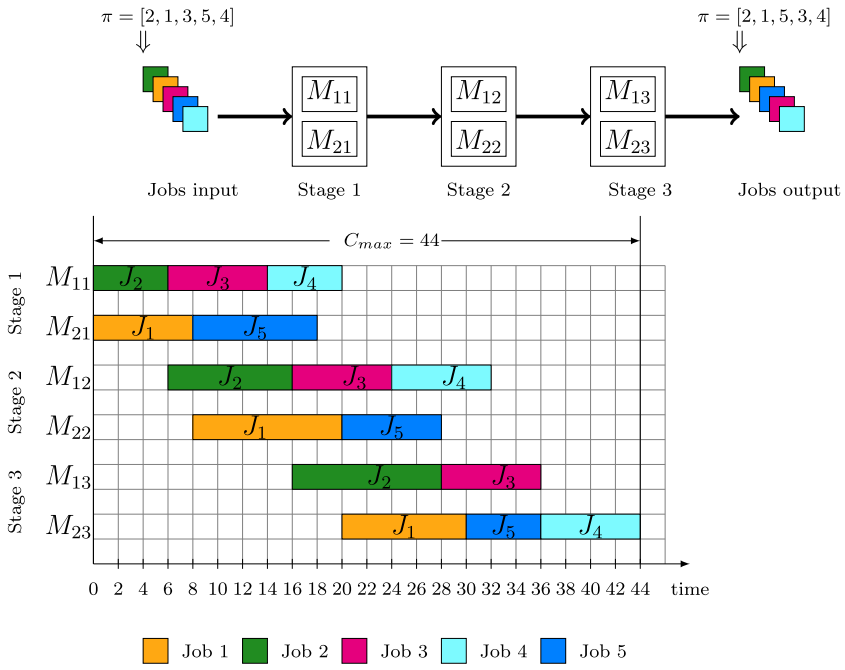



**Fig. 3.** Gantt chart for HFS with three stages.

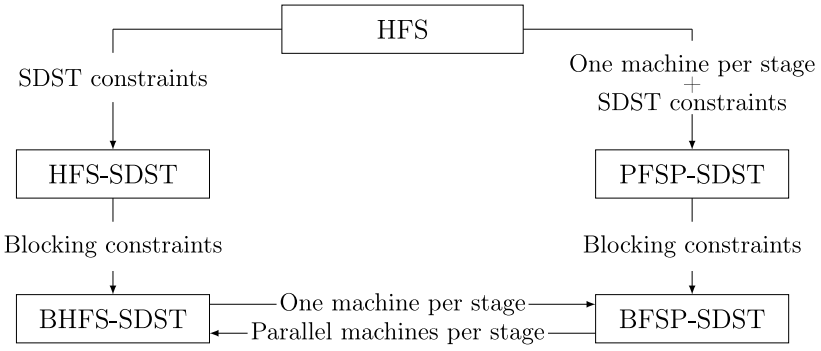



**Fig. 4.** Connection diagram between BHFS-SDST and BFSP-SDST.

the makespan of an HFS-SDST problem. In Khalouli et al. (2010), we find an ant colony optimization (ACO) algorithm to minimize the total weighted tardiness and earliness of the problem HFS. Zabihzadeh and Rezaeian (2016) proposed a MILP, ACO algorithm and GA procedure to solve an HFS problem under transport constraints and realize dates. In Fernandez-Viagas et al. (2019), a model of MILP for the HFS problem with identical parallel machines is developed to minimize the makespan.

Similarly, in Ernst et al. (2019) a modelization in MILP form is proposed for the case of two stages with the constraint of the buffer capacity. In Kurdi (2019), an improved version of ACO and a model in MILP are proposed in order to solve the HFS problem. González-Neira and Montoya-Torres (2017) suggested the GRASP to minimize many tardiness criteria for HFS manufacturing production. In Öztop et al. (2019), a MILP and a various version on the IG algorithm are proposed to solve the HFS problem in order to minimize total flow time criterion. In Pan et al. (2017), a set of the ILS, IG algorithms are developed to minimize the weighted tardiness and earliness, from the due window, to solve an HFS problem. Similar metaheuristics involving ILS and IG are used to solve slabs and beams HFS-SDST manufacturing problem (Aqil and Allali, 2020). In Zohali et al. (2019), a MILP formulation as well as a hybrid version in ILS are proposed to minimize the cost of the HFS problem with economic lot-sizing. In Wang et al. (2019), a set of hybrid variable neighborhood search (VNS) algorithms is developed to solve an HFS scheduling process with distribution constraint. In Valizadeh et al. (2019), the algorithm of particle swarm optimization (PSO) is considered to solve the flexible flow line problem to minimize the makespan the total flow time with machine eligibility.

The complexity of manufacturing system and the large size of new industrial problems in the presence of new constraints make it increasingly difficult to propose exact approaches. The use of the approximate methods is very interesting. Indeed the heuristics and the metaheuristics become a necessity to find good solutions of the studied problems. This pushes researchers to develop new methods that can combine several algorithms forming hybrid metaheuristics. In addition, the development of new artificial intelligence algorithms makes it possible to implement new metaheuristics inspired by the behavior of living beings or natural phenomena. In Pan and Dong (2014) and Zhang et al. (2017), a resolution based on the MBO algorithm is discussed to solve the HFS and the HFS with lot-streaming problems respectively, in order to minimize the total flow time. Research is continuing in the area of optimization, in particular based on metaheuristics inspired by the behavior of animals or phenomena resulting from nature. Recently, in Khare and Agrawal (2019), the authors propose three hybrids metaheuristic: squirrel search algorithm (SSA), whale optimization algorithm (WOA) and discrete grey wolf optimization (DGWO) to solve the HFS problem. We are suggesting new versions of two hybrid metaheuristics, based

**Table 2**
The used notation of BHFS-SDST problem.

| Notation | Description |
|---|---|
| $n$ | Numbers of jobs |
| $L$ | Numbers of stages |
| $l$ | Index of stage, $l = 1, 2, \dots, L$ |
| $i$ | Index for machines in stage $l$, $i = 1, 2, \dots, m_l$ |
| $j, k$ | Index of jobs, $j = 1, 2, \dots, n$, $k = 1, 2, \dots, n$ |
| $r_j$ | The realize date of job $j$ |
| $d_j$ | The due date of job $j$ |
| $p_{jl}$ | The processing time of job $j$ in stage $l$ |
| $v_{il}$ | The speed of machine $i$ at stage $l$ |
| $s_{kjl}$ | The sequence dependent setup time when job $k$ precedes job $j$ at stage $l$ |
| $C_{jil}$ | The completion time of job $j$ in machine $i$ at stage $l$ |
| $Ct_{jl}$ | The completion time of the job $j$ at stage $l$ |
| $D_{jl}$ | The starting time of job $j$ on the stage $l$ |
| $J$ | The set of $n$ jobs, $J = \{J_1, J_2, \dots, J_n\}$ |
| $M^{(l)}$ | The set of machines at stage $l$, $M^{(l)} = \{M_{1l}, M_{2l}, \dots, M_{m_l l}\}$ |
| $m_l$ | Number of machines at stage $l$ |

**Table 3**
The used notation in coding the objective function.

| Notation | Description |
|---|---|
| $\pi$ | A given sequence schedule, where $\pi = [\pi(1), \pi(2), \dots, \pi(n)]$ |
| $j$ | Index of position in the sequence $\pi$, $j = 1, 2, \dots, n$ |
| $r_{\pi(j)}$ | The realize date of job at position $j$ in the sequence $\pi$ |
| $d_{\pi(j)}$ | The due date of job at position $j$ in the sequence $\pi$ |
| $p_{\pi(j)l}$ | The processing time of job at position $j$ in the sequence $\pi$ in stage $l$ |
| $s_{\pi(k)\pi(j)l}$ | The sequence dependent setup time when job at position $k$ precedes job at position $j$ in the sequence at stage $l$ |
| $C_{\pi(j)il}$ | The completion time of job at position $j$ in the sequence $\pi$ in machine $i$ at stage $l$ |
| $Ct_{\pi(j)l}$ | The completion time of the job at position $j$ in the sequence $\pi$ on stage $l$ |
| $D_{\pi(j)l}$ | The starting time of job $j$ at position $j$ in the sequence $\pi$ on the stage $l$ |
| $T_{\pi(j)}$ | The tardiness of job at position $j$ in the sequence $\pi$ |
| $E_{\pi(j)}$ | The earliness of job at position $j$ in the sequence $\pi$ |
| $Z$ | The objective function value |
| $U_{il}$ | The memory of machine $i$ at stage $l$ |

on MBO and WWO algorithms. This contribution consists in proposing three versions for each metaheuristic solving the BHFS-SDST manufacturing. The algorithms are consolidated with different local search procedures to improve our numerical methods. The modification in the new algorithms concerns the phases of the neighborhood exploration and the way of the convergence of our metaheuristics toward their good solutions. The objective function is to minimize the total tardiness and earliness taking into account other constraints relating to the realize and the due dates of jobs. Our model is inspired by an industrial case of painting shop manufacturing (Lee et al., 2019). The proposed model concerns a production extended to several stages and to several machines on each stage. Such approach constitutes a flexible version that can be adapted to many types of production of the same type of flow.

We give in Table 2 the adopted notation of the used parameters and variables of our problem.

## 3. The BHFS-SDST scheduling problem

### 3.1. Problem statement

The BHFS-SDST scheduling problem is represented by a succession of $L$ stages where on each stage a set of parallel machines with proportional performances can fulfill the operation of the available job. We denote $J = \{J_1, J_2, \dots, J_n\}$ the jobs forming the batch production to be executed in the workshop process. Each job will be treated by a single machine of the stage; all jobs starting with the first stage and finishing the process on the last stage. We note, $M^{(l)} = \{M_{1l}, M_{2l}, \dots, M_{m_l l}\}$ a set forming the machine park of on each stage $l$ the manufacturing process. All the machines are UPM is capable of processing the same job during scheduling, knowing that $|M^{(l)}| = m_l$.

The processing time of the job $J_j$ on the stage $l$ is called $p_{jl}$. Each job $J_j$ has a realize date $r_j$ characterizing its availability in front of the first stage and a date due $d_j$ on the last stage. Each machine $M_{il}$ of the stage $l$ has a speed $v_{il}$. The $s_{kjl}$ designates the machine's setup time when the $J_k$ precedes the job $J_j$ in the stage $l$ when scheduling processes in the factory. If the job $J_j$ is the first on its machine, this time will be designated by $s_{jjl}$. The completion time of the job $J_j$ by the machine $i$ of the stage $l$ is $C_{jil}$, when the machine is not specified the completion time will be denoted $Ct_{jl}$. Taking into account the blocking constraint, the starting date of the job $J_j$ the stage $l$ is designated $D_{jl}$. This date is also the start date of treatment in the $l + 1$ stage. The lack of space in the workshop and the nature of the jobs carried out impose the constraint of blocking jobs in the machines. Not all jobs are available at the same time; all the machines are available at the initial moment. We retain a set of other general hypotheses in the scheduling of HFS workshops. A job will only be processed by one machine on each stage and a machine can only perform one job at a given time. A sequence $\pi = [\pi(1), \pi(2), \dots \pi(n)]$ is a permutation of jobs started in the manufacturing process. During scheduling at the end of each stage the jobs will be sorted in ascending order from their end date to the current stage to be processed by the next stage.

We are extending the real model for the multistage HFS problem. The aim is to propose a global model for a possible extension of our approach to other additional stages. In Fig. 5, the applied scheduling in the workshop is shown for the HFS manufacturing problem without inter-stage buffer storage.

### 3.2. Coding the objective function

To give the expression of the objective function of the BHFS-SDST problem, we describe a set of variables and parameters needed for the calculation. We consider that $T_{\pi(j)} = max(Ct_{\pi(j)L} - d_{\pi(j)}, 0)$ and $E_{\pi(j)} = max(d_{\pi(j)} - Ct_{\pi(j)L}, 0)$ represent the tardiness and the earliness of jobs $\pi(j)$ during its scheduling respectively. At the end of the processing of the entire batch of jobs, we can evaluate the total tardiness and earliness by the objective function $Z = \sum_{j=1}^{n}(E_{\pi(j)} + T_{\pi(j)})$. In Algorithm 1, we present the different steps in solving BFSH-SDST scheduling problem. In this procedure, we consider the blocking constraints as well as that of the SDST. This makes it possible to determine the allocation, the sequencing as well as the scheduling of the manufacturing process of each job in the scheduled sequence. In Table 3, we give the used parameters and variables coding function objective to minimize in BHFS-SDST.

To be able to determine the starting dates of the jobs on each stage, we use the variable $U_{il}$ which allows storing the jobs assigned to each machine. This variable is required to memorize the best machine on the next stage $l$ of each job. This, therefore, allows to know the completion time $Ct_{\pi(j)l}$ and the departure time of the job $\pi(j)$ of the stage $(l-1)$ is $D_{\pi(j)(l-1)}$. Two cases arise; if $Ct_{\pi(j)l} \leqslant Ct_{\pi(j)(l-1)}$ in this case the job $\pi(j)$ finds its best available machine on stage $l$ without blocking and $D_{\pi(j)(l-1)} = Ct_{\pi(j)(l-1)}$. Otherwise, $D_{\pi(j)(l-1)} = Ct_{\pi(j)l} - p_{\pi(j)l}/v_{i^*l}$, in which case the job $\pi(j)$ will be blocked in its previous stage machine despite its completion rather until the availability of its best machine to the next stage. In the last stage the departure date of each job is equal to its completion time, that is, $D_{\pi(j)L} = Ct_{\pi(j)L}$.

We note that in the first phase, we proceed to the scheduling in the first stage. Later in the second phase we order the jobs of the sequence in ascending order of their end date $Ct_{\pi(j)(l-1)}$ from the previous stage. When assigning the job $\pi(j)$ to the stage $l$ we will select the best one that will complete it as soon as possible; that is $i^* = argmin_{i \in M^l}\{C_{\pi(j)il}\}$. The process repeats until all the jobs in sequence are completed in all stages.

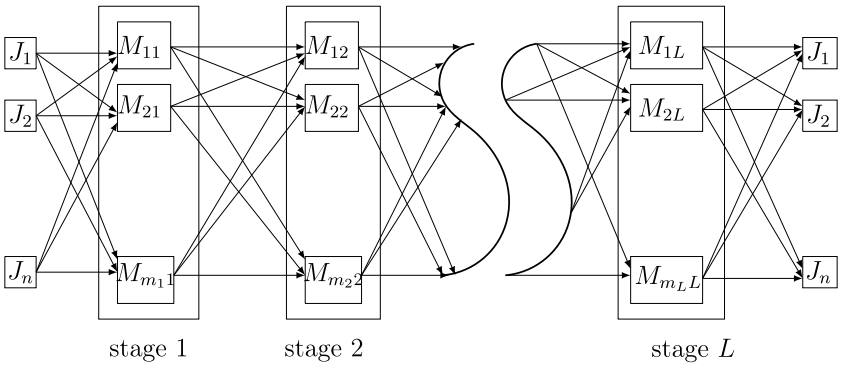



**Fig. 5.** $L$ stages hybrid flow shop.

**Algorithm 1:** Procedure function

**input** : $\pi_0$ intial, $U_{il} = \emptyset, l = 1, ..., L$
$\pi \leftarrow \pi_0, \pi = [\pi(1), \pi(2), ...\pi(n)]$
***Step 1:*** *scheduling in the first stage*
**for** $j = 1$ *to* $n$ **do**
  **for** $i = 1$ *to* $m_1$ **do**
    **if** $U_{i1} = \emptyset$ **then**
      $C_{\pi(j)i1} = max(s_{\pi(j)\pi(j)1}, r_{\pi(j)}) + p_{\pi(j)1}/v_{i1}$
    **else**
      $C_{\pi(j)i1} = C_{\pi(k)1} + s_{\pi(k)\pi(j)1} + p_{\pi(j)1}/v_{i1}$ % $\pi(k)$ precedes $\pi(j)$ in the machine $i$%
    **end**
    $i^* = \text{argmin}_{i \in M^1}\{C_{\pi(j)i1}\}$, $Ct_{\pi(j)1} \leftarrow C_{\pi(j)i^*1}$
    $U_{i^*1} \leftarrow U_{i^*1} \bigcup \pi(j)$ % update the memory machines%
  **end**
**end**
***Step 2:*** *scheduling in the other stages*
**for** $l = 2$ *to* $L$ **do**
  *Order the jobs in ascending order of* $Ct_{\pi(j)(l-1)}$ *and build the sequence* $\pi = [\pi(1), \pi(2), ..., \pi(n)]$
  **for** $j = 1$ *to* $n$ **do**
    **for** $i = 1$ *to* $m_l$ **do**
      **if** $U_{il} = \emptyset$ **then**
        $C_{\pi(j)il} = max(Ct_{\pi(j)(l-1)}, s_{\pi(j)\pi(j)l}) + p_{\pi(j)l}/v_{il}$
      **else**
        $C_{\pi(j)il} = max(Ct_{\pi(j)(l-1)}, Ct_{\pi(k)l} + s_{\pi(k)\pi(j)l}) + p_{\pi(j)l}/v_{il}$ % $\pi(k)$ precedes $\pi(j)$ in the machine $i$ at stage $l$%
      **end**
      $i^* = argmin_{i \in M^l}\{C_{\pi(j)il}\}$
      $Ct_{\pi(j)l} \leftarrow C_{\pi(j)i^*l}$, $U_{i^*l} \leftarrow U_{i^*l} \bigcup \pi(j)$
      $D_{\pi(j)(l-1)} \leftarrow max(Ct_{\pi(j)(l-1)}, Ct_{\pi(j)l} - p_{\pi(j)l}/v_{i^*l})$
    **end**
  **end**
**end**
**for** $j = 1$ *to* $n$ **do**
  $D_{\pi(j)L} \leftarrow Ct_{\pi(j)L}$
**end**
***Step 3:*** *Evaluation of the objective function*
$Z \leftarrow 0$
**for** $j = 1$ *to* $n$ **do**
  $Z \leftarrow Z + max(D_{\pi(j)L} - d_\pi(j), 0) + max(d_\pi(j) - D_{\pi(j)L}, 0)$
**end**
**output**: Z

### 3.3. Numerical illustration

The following numerical example illustrates a scheduling case studied in the BHFS-SDST problem. We want to schedule a sequence of $n = 8$ jobs, on $L = 4$ stages, with the following set of machines: $M^{(1)} = \{M_{11}, M_{21}\}, M^{(2)} = \{M_{12}, M_{22}, M_{32}\}, M^{(3)} = \{M_{13}, M_{23}, M_{33}\}, M^{(4)} = \{M_{14}, M_{24}\}$, the sequence is $\pi = [7, 4, 1, 3, 8, 5, 2, 6]$. Tables 4–6 and the matrix's $[s_{kj1}], [s_{kj2}], [s_{kj3}], [s_{kj4}]$ represent the data of the instance.

$$[s_{kj1}] = \begin{bmatrix} 5 & 3 & 1 & 2 & 3 & 2 & 3 & 2 \\ 2 & 1 & 2 & 1 & 1 & 2 & 3 & 3 \\ 3 & 2 & 5 & 1 & 2 & 1 & 3 & 6 \\ 2 & 1 & 4 & 2 & 3 & 3 & 3 & 4 \\ 5 & 2 & 2 & 1 & 4 & 4 & 3 & 4 \\ 3 & 4 & 4 & 1 & 3 & 1 & 3 & 1 \\ 4 & 3 & 2 & 1 & 2 & 3 & 3 & 3 \\ 1 & 4 & 5 & 1 & 1 & 2 & 3 & 2 \end{bmatrix};$$

$$[s_{kj2}] = \begin{bmatrix} 3 & 4 & 4 & 2 & 2 & 5 & 4 & 3 \\ 2 & 1 & 4 & 5 & 5 & 4 & 3 & 5 \\ 2 & 4 & 5 & 2 & 3 & 2 & 2 & 5 \\ 3 & 4 & 2 & 3 & 3 & 1 & 3 & 6 \\ 4 & 4 & 5 & 3 & 2 & 4 & 4 & 6 \\ 2 & 3 & 3 & 5 & 2 & 3 & 2 & 1 \\ 2 & 5 & 1 & 5 & 5 & 4 & 3 & 2 \\ 2 & 4 & 4 & 5 & 4 & 2 & 1 & 3 \end{bmatrix};$$

$$[s_{kj3}] = \begin{bmatrix} 5 & 3 & 1 & 2 & 3 & 2 & 3 & 3 \\ 2 & 1 & 2 & 1 & 1 & 2 & 3 & 3 \\ 3 & 6 & 5 & 1 & 2 & 3 & 5 & 6 \\ 2 & 1 & 3 & 5 & 3 & 3 & 3 & 4 \\ 5 & 2 & 2 & 1 & 4 & 4 & 3 & 2 \\ 2 & 4 & 4 & 1 & 3 & 1 & 3 & 1 \\ 4 & 3 & 2 & 1 & 3 & 4 & 3 & 3 \\ 3 & 4 & 5 & 1 & 1 & 2 & 3 & 2 \end{bmatrix};$$

$$[s_{kj4}] = \begin{bmatrix} 2 & 4 & 4 & 2 & 4 & 5 & 4 & 3 \\ 2 & 1 & 4 & 5 & 5 & 4 & 3 & 5 \\ 2 & 4 & 5 & 2 & 3 & 2 & 2 & 5 \\ 3 & 4 & 3 & 1 & 3 & 1 & 3 & 6 \\ 4 & 4 & 5 & 3 & 2 & 2 & 4 & 6 \\ 2 & 3 & 3 & 5 & 2 & 3 & 2 & 1 \\ 2 & 5 & 1 & 5 & 3 & 4 & 3 & 2 \\ 2 & 4 & 4 & 5 & 4 & 2 & 1 & 3 \end{bmatrix}$$

The Gantt chart of the scheduling solution for this instance is shown in Fig. 6. Each job is scheduled on its best machine on each stage. We can notice that the first jobs $J_7$ and $J_4$ will be scheduled taking into account their realize times $r_7 = 4$ and $r_4 = 3$. After that, the other jobs are launched in the planning cycle. We note that the blocking constraint requires that the job only leaves stage $l - 1$ if its best machine on stage $l$ is not available.

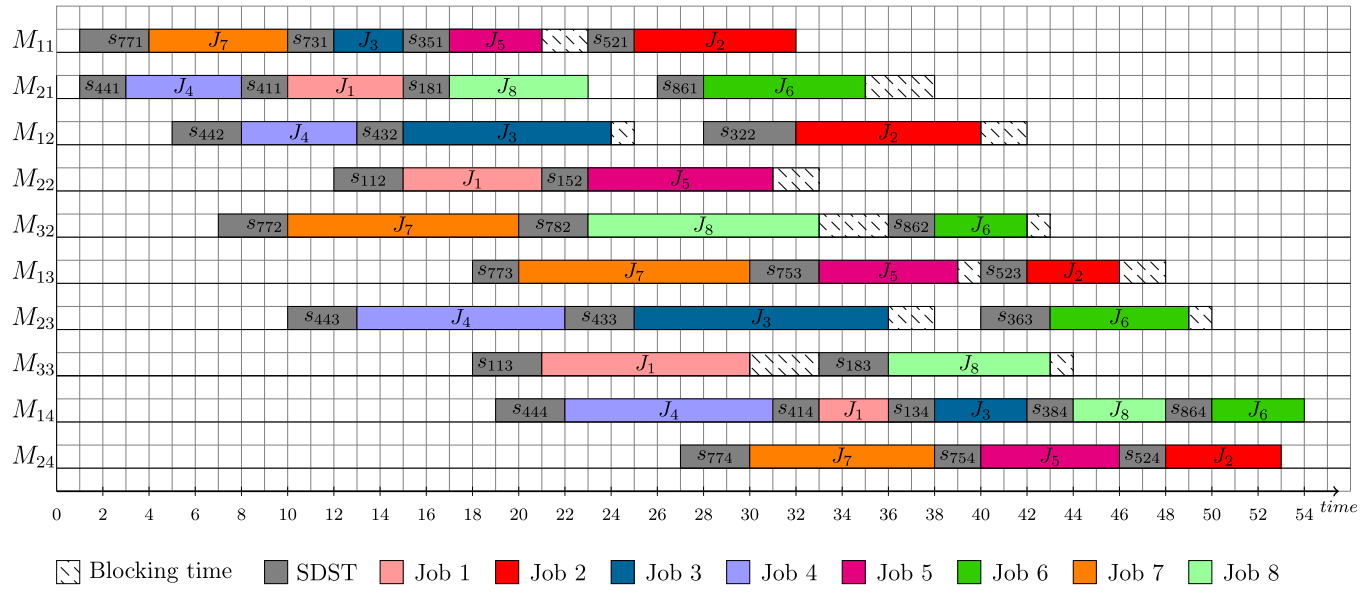



**Fig. 6.** Gantt chart for eight jobs in four stages.

**Table 4**
Realize and due date of jobs.

| | $J_1$ | $J_2$ | $J_3$ | $J_4$ | $J_5$ | $J_6$ | $J_7$ | $J_8$ |
|---|---|---|---|---|---|---|---|---|
| $r_j$ | 6 | 8 | 5 | 3 | 8 | 12 | 4 | 10 |
| $d_j$ | 40 | 48 | 38 | 34 | 44 | 50 | 40 | 42 |

**Table 5**
Processing time of jobs.

| Job | | $J_1$ | $J_2$ | $J_3$ | $J_4$ | $J_5$ | $J_6$ | $J_7$ | $J_8$ |
|---|---|---|---|---|---|---|---|---|---|
| Stage | 1 | 5 | 14 | 6 | 5 | 8 | 7 | 12 | 6 |
| | 2 | 6 | 16 | 18 | 10 | 8 | 6 | 15 | 15 |
| | 3 | 9 | 6 | 20 | 18 | 9 | 12 | 15 | 7 |
| | 4 | 6 | 5 | 8 | 18 | 6 | 8 | 8 | 8 |

**Table 6**
Speed of all machines.

| | Stage 1 | | Stage 2 | | | Stage 3 | | | Stage 4 | |
|---|---|---|---|---|---|---|---|---|---|---|
| | $M_{11}$ | $M_{21}$ | $M_{12}$ | $M_{22}$ | $M_{32}$ | $M_{13}$ | $M_{23}$ | $M_{33}$ | $M_{14}$ | $M_{24}$ |
| $v_{il}$ | 2 | 1 | 2 | 1 | 1.5 | 1.5 | 2 | 1 | 2 | 1 |

**Table 7**
Results of the instance.

| | $J_1$ | $J_2$ | $J_3$ | $J_4$ | $J_5$ | $J_6$ | $J_7$ | $J_8$ |
|---|---|---|---|---|---|---|---|---|
| $Ct_{j1}$ | 15 | 32 | 15 | 8 | 21 | 35 | 10 | 23 |
| $D_{j1}$ | 15 | 32 | 15 | 8 | 23 | 38 | 10 | 23 |
| $Ct_{j2}$ | 21 | 40 | 24 | 13 | 31 | 42 | 20 | 33 |
| $D_{j2}$ | 21 | 42 | 25 | 13 | 33 | 43 | 20 | 36 |
| $Ct_{j3}$ | 30 | 46 | 36 | 22 | 39 | 49 | 30 | 43 |
| $D_{j3}$ | 33 | 48 | 38 | 22 | 40 | 50 | 30 | 44 |
| $Ct_{j4}$ | 36 | 53 | 42 | 31 | 46 | 54 | 38 | 48 |
| $D_{j4}$ | 36 | 53 | 42 | 31 | 46 | 54 | 38 | 48 |
| $E_j$ | 4 | 0 | 0 | 3 | 0 | 0 | 2 | 0 |
| $T_j$ | 0 | 5 | 4 | 0 | 2 | 4 | 0 | 6 |

In addition, a job remains blocking in its machine and the setup time of the machine for the new job will only be carried out if the current job has left it. Table 7 summarizes the results obtained; we will mainly identify the completion and departure times of each job in the four stages. According to the scheduling calendar, we can see the blocking times for the concerned jobs. In addition, we calculate the earliness $E_j$ and the tardiness $T_j$ of each job as well as the value of the objective function $Z = \sum_{j=1}^{8}(E_j + T_j) = 30$ units of time.

## 4. The resolution approaches

### 4.1. The proposed algorithms

The proposed approaches to solve BHFS-SDST problem are classified into two main MBO and WWO algorithms. We suggest three versions for each of them and we introduce changes in the local search phase. These search methods are based on the neighborhoods exploration of current solution. The improvement is based on a new neighborhood generation approach by given a set of neighboring solutions from which we choose the best one. We develop two new versions of neighborhood generations by inserting and permuting elements to generate a neighborhood set. The third enhanced procedure (EP) is based on the GRASP constructive algorithm for neighborhood exploration. We also apply the iterated greedy algorithm, as well as that of the path re-linking (PR) in the neighborhood exploration phases of the WWO algorithm.

### 4.2. The MBO algorithm

The MBO algorithm is one of the effective meta-heuristic optimization. This nature inspired method, based on the behavior of migratory birds in flight. It was implemented for the first time in Duman et al. (2012) for solving the quadratic assignment optimization problem. Subsequently several applications of this metaheuristic succeeded in the optimization scheduling problems. In Benkalai et al. (2017) and Sioud and Gagné (2018) the MBO algorithm is proposed to solve the permutation flow shop scheduling problem (PFSP) under constraint SDST to minimize the makespan. In Algorithm 2, we give our version of the MBO algorithm to solve the BHFS-SDST problem.

We note that the principle of this algorithm is to divide a population $\Pi$ whose number of elements $\eta = |\Pi|$ of solutions into two sub-populations one to the right $\Pi_R$ with $|\Pi_R| = (\eta - 1)/2$ and the other one to the left $\Pi_L$ with $|\Pi_L| = (\eta - 1)/2$. As long as a stop criterion is not satisfied and the number of turns $\omega$ is not reached, each solution is disturbed according to a neighborhood ($\mathcal{N}_i$) in order to improve the leader solution. We propose an enhanced migratory birds optimization (EMBO) algorithm. We are developing EP with two versions based on local search (LS) and a version based on the GRASP constructive algorithm. Finally, we define three versions of the EMBO algorithm according to the three EP techniques (see Table 8). The used notation in our algorithms is presented in Table 8, Table 9, Table 10, Table 11, Table 12 and Table 13.

**Table 8**
The used notation in the EMBO algorithm.

| Notation | Description |
|---|---|
| $\Pi$ | Population of sequence in EMBO algorithm |
| $\Pi_R$ | Sub-population in the right |
| $\Pi_L$ | Sub-population in the left |
| $\omega$ | The number of turns |
| $\eta$ | The cardinal of the population |
| $\pi$ | The current sequence |
| $\tilde{\pi}$ | The best obtained sequence by EP |
| $\pi^r$ | The best sequence in $\Pi_R$ |
| $\pi^l$ | The best sequence in $\Pi_L$ |
| $\tilde{\pi}$ | The best sequence given by EP |
| $\mathcal{N}_i$ | Neighborhood structure, $i = 1, 2, \ldots, 4$ |
| $\pi^*$ | The best sequence in the EMBO algorithm |

**Algorithm 2:** EMBO algorithm

```
Step 1: Generate the initial population and divide it into two
sub-populations Π_R, Π_L and deduce the best sequence π*, π ← π*
Step 2
while Stopping criterion is not satisfied do
    for k = 1 to ω do
        π̃ The best obtained sequence by EP applied to π
        if Z(π̃) ⩽ Z(π*) then
            π* ← π̃
        end
        for r = 1 to |Π_R| do
            π' ← N_i(π^r)
            if Z(π') ⩽ Z(π^r) then
                π^r ← π'
                if Z(π^r) ⩽ Z(π*) then
                    π* ← π^r
                end
            end
        end
        for l = 1 to |Π_L| do
            π' ← N_i (π^l)
            if Z(π') ⩽ Z(π^l) then
                π^l ← π'
                if Z(π^l) ⩽ Z(π*) then
                    π* ← π^l
                end
            end
        end
    end
    π is obtained by disrupting π* using N_i neighborhood
end
output: Z*
```

Moreover, to find a better solution by exploring the neighborhood, we apply the methods based on the permutation and insertion techniques of job positions in the scheduling sequence. In this procedure, given two positions $p$ and $q$, the principle consists in:

- $\mathcal{N}_1$: swap jobs $\pi(p)$ and $\pi(q)$.
- $\mathcal{N}_2$: if $p < q$ insert $\pi(p)$ in position $q$ and shift the left.
- $\mathcal{N}_3$: if $p \geqslant q$ insert $\pi(p)$ in position $q$ and shift the right.
- $\mathcal{N}_4$: inverse the block of jobs between $\pi(p)$ and $\pi(q)$.

#### 4.2.1. Enhanced procedure with local search

In this study, our modified approach for the EMBO algorithm is characterized by two local search EP versions named $LS_1$ and $LS_2$ according to two different neighborhoods $N_1$ and $N_2$, respectively. The first $LS_1$ allows full exploration by inserting and shifting one of the two sub-sequences $\pi^1$ or $\pi^2$ to get the set $\Gamma_1^{N_1}$ or $\Gamma_2^{N_1}$. We complete the built sequence by inserting the remaining missing jobs. The second $LS_2$ is based on the set of solutions $\Gamma_1^{N_2}$ or $\Gamma_2^{N_2}$ by total permutation. In the same way to complete the constructed sequence, we insert the rest of the missing jobs in a random position. In Algorithm 3, we propose an implementation version of the two $LS_u$, $(u = 1, 2)$ local search enhancement procedures.

**Table 9**
The used notation in the local search procedure.

| Notation | Description |
|---|---|
| $LS_u$ | Local search procedure, $u = 1, 2$ |
| $\pi = [\pi^1\ \pi^2]$ | The currents sequence decomposed in two sub sequence $\pi^1$ and $\pi^2$ |
| $N_u$ | Neighborhood procedure with local search $LS_u$, $u = 1, 2$ |
| $\Gamma_u^{N_1}$ | The set of sub-sequence generate by $LS_1$, applied to $\pi^u$, $u = 1, 2$ |
| $\Gamma_u^{N_2}$ | The set of sub-sequence generate by $LS_2$, applied to $\pi^u$, $u = 1, 2$ |
| $\pi_i^1$ | The sub-sequence in set $\Gamma_u^{N_1}$ |
| $\pi_i^2$ | The sub-sequence in set $\Gamma_u^{N_2}$ |
| $\tilde{\pi}$ | The current solution built |
| $\pi_{best}$ | The best sequence in $LS_u$ procedure |

To illustrate the neighborhood EP model $LS_1$, we propose in Fig. 7 an example showing the developed approach. The method consists, therefore, in generating a random number $\theta$ according to a uniform law $\theta \in [0, 1]$ if $\theta \leqslant 0.5$. The current built sequence is the concatenation of two sub-sequences $\pi_i^1$ and $\pi^2$ such that $\bar{\pi} = [\pi_i^1\ \pi^2]$, where $\pi_i^1 \in \Gamma_1^{N_1}$.

In the opposite case, the current constructed sequence is obtained by the concatenation of the two sub-sequences $\pi_i^1$ and $\pi^2$ such that $\bar{\pi} = [\pi_i^2\ \pi^1]$, such as $\pi_i^2 \in \Gamma_2^{N_1}$. To illustrate these two EP of neighborhoods, we consider the sequence $\pi = [7, 1, 4, 8, 3, 2, 6, 5]$, given the two sub-sequences $\pi^1 = [3, 4, 6, 5]$ and $\pi^2 = [8, 1, 7, 2]$. We note that if $\mu_1 = |\pi^1|$ then the number of solutions generated by this procedure is $(\mu_1 - 1)^2$. We consider that the value can vary, such as $\mu_1 \in \{n/3, n/2, 2n/3\}$. The goal is to better explore the neighborhood for the current solution (see Table 9).

**Algorithm 3:** Local search procedure $LS_u$, $(u = 1, 2)$

```
π (The current best solution)
Decompose π in two sub-sequences π = [π^1 π^2] and generate sets
Γ_1^{N_u} Γ_2^{N_u} according to the type of neighborhood N_u chosen
if θ < 0.5 then
    for i = 1 to |Γ_1^{N_u}| do
        Evaluate the Z function of each sequence obtained by the
        concatenation π̄ = [π_i^1 π^2] where π_i^1 ∈ Γ_1^{N_u} and remember
        the best sequence and retaining the best sequence π_best
    end
else
    for i = 1 to |Γ_2^{N_u}| do
        Evaluate the Z function of each sequence obtained by the
        concatenation π̄ = [π_i^2 π^1] where π_i^2 ∈ Γ_2^{N_u} and remember
        the best sequence and retaining the best sequence π_best
    end
end
if Z(π_best) ⩽ Z(π) then
    π ← π_best
end
```

In the same way, to illustrate the EP approach by $LS_2$ (see Fig. 8), starting from the current sequence $\pi = [7, 1, 4, 8, 3, 2, 6, 5]$, and dividing this sequence into two sub-sequences $\pi^1 = [3, 4, 6, 5]$ and $\pi^2 = [8, 1, 7, 2]$. We generate the neighborhood structure $\Gamma_u^{N_2}$ by total permutation without repetition. We can see that if $\mu_2 = |\pi^1|$ then the number of generated solutions by this method is $\mu_2 \times (\mu_2 - 1)/2$.

#### 4.2.2. The enhanced procedure GRASP

In order to improve the MBO algorithm, we also propose the GRASP procedure used in Davoudpour and Ashrafi (2009) and Alekseeva et al.

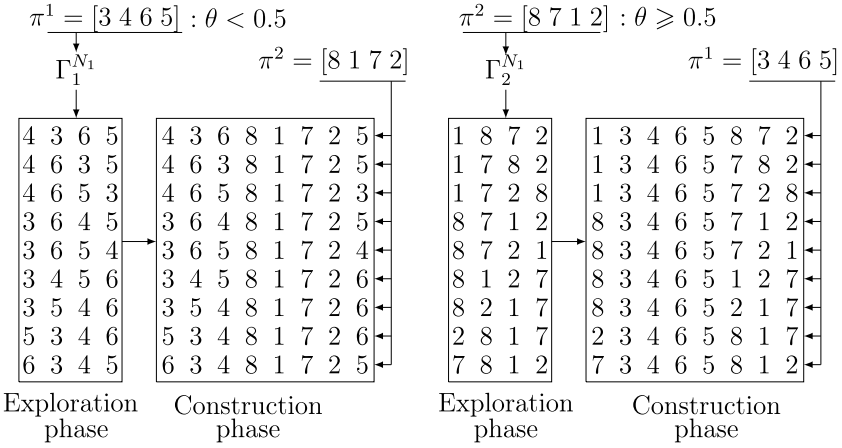



**Fig. 7.** $LS_1$ neighborhood exploration.

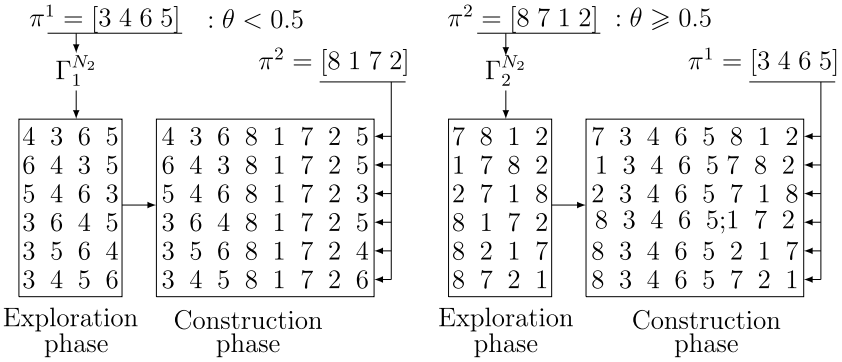



**Fig. 8.** $LS_2$ neighborhood exploration.

(2017) with local search methods. This procedure (see Algorithm 4) has been well applied to solve scheduling problems because of these reduced parameters and its ease of implementation. Starting from an empty solution, the approach consists in selecting a job and inserting it in different positions and retaining the best sequence $\pi^{gr}$. In this constructive approach to solutions, the process is repeated until all the jobs in the sequence to be explored are completed. It is based on two essential steps: the construction phase and the second phase with the local search. In our approach we have limited ourselves here to the first constructive step. In this step, considering an empty starting solution, at each iteration, a set of jobs is selected from a restricted candidate list (RCL) of jobs. This list is defined according to the largest and the smallest value of the objective function. The choice of the elements interval in the RCL list is defined by the Eq. (1)

$$Z(\pi(j)) \in [Z_1, Z_1 + \alpha \times (Z_2 - Z_1)] \tag{1}$$

We specify that $Z(\pi(j))$ denotes the value of the objective function obtained by insertion of the $\pi(j)$ job in the build sequence. The terms $Z_1$ and $Z_2$ designate the minimum and maximum values of the objective function of the current sequence build , respectively. This condition limits the choice to take only some job that can be scheduled on the sequence being built. The coefficient $\alpha \in [0.3, 0.7]$ is a parameter that allows increasing or decreasing the selection range of the RCL list (see Table 10).

We can see that the implementation of this procedure requires knowledge of two variants: the parameter $\alpha$ and the method of choosing the candidate element of the list RCL. Our EMBO algorithm resolution

**Table 10**
The used notation in GRASP procedure.

| Notation | Description |
|---|---|
| $\pi$ | The currents sequence |
| $\pi^{gr}$ | The best sequence given by GRASP procedure |
| $\phi$ | The set of sub-sequence generate by GRASP procedure |
| $Z_1$ | The minimal value in RCL list |
| $Z_2$ | The maximal value in RCL list |
| $\alpha$ | The GRASP parameter |

**Algorithm 4:** The GRASP enhanced procedure

**input** : $:\pi = [\pi(1), .., \pi(n)]$ (The current best solution)
$\pi^{gr} \leftarrow \emptyset$, % $\pi^{gr}$ *the GRASP sequence%*
$\phi \leftarrow \pi$, *and evaluate the function* $Z$ *of the* $\pi(j) \in \phi$
**while** $\phi \neq \emptyset$ **do**
  $Z_1 \leftarrow min\{Z(\pi(j)),\ \pi(j) \in \phi\}\ Z_2 \leftarrow max\{Z(\pi(j)),\ \pi(j) \in \phi\}$
  $\delta Z = Z_2 - Z_1$
  *Build the* $RCL = \{\pi(j),\ \ Z(\pi(j)) \in [Z_1, Z_1 + \alpha \times \delta Z]\}$
  *Select a sequence* $\pi(u)$ *from the RCL and choose the best position giving the minimum function value* $Z$ *in the constructed solution*
  $\pi^{gr} \leftarrow \pi^{gr} \cup \pi(u)$ and Update from $\phi$
**end**
**output**: $:\pi^{gr}$

approach is developed according to three versions $EMBO_i,\ i = 1, 2, 3$. The first two versions are obtained by applying the enhancement procedures based on $LS_u,\ u = 1, 2$; the third version is obtained by the implementation of GRASP method. However, we propose the same

**Table 11**
The used notation in WWO algorithm.

| Notation | Description |
|---|---|
| $\Pi$ | Population of sequence in WWO algorithm |
| $N_w$ | The size of Population $\Pi$ |
| $\pi^i$ | The current sequence in $\Pi$, $i = 1, 2, \ldots, N_w$ |
| $\lambda$ | The wavelength |
| $H_{max}$ | The maximum amplitude |
| $H$ | The current amplitude |
| $\pi^{best}$ | The best sequence in $\Pi$ |
| $\pi^{worst}$ | The worst sequence in $\Pi$ |
| $\pi^p$ | The sequence given by DCP procedure |
| $\pi'$ | The sequence given by neighborhood applied to $\pi^p$ |

parameter settings for the three proposed algorithms, such as the population size and the maximum number of rounds $\omega$.

### 4.3. WWO algorithm

#### 4.3.1. Presentation

The water wave optimization algorithm (Zheng, 2015) is among the algorithms inspired by the natural phenomena applied in the optimization field. This metaheuristic is one of the most powerful algorithms for solving optimization problems. Recently, in Zhao et al. (2018) WWO is implemented to solve the no wait PFSP. Similarly, in Zhao et al. (2019) proposed a hybrid version of the WWO algorithm to solve the no idle PFSP. The pseudo-code presented in Algorithm 5 shows the implemented steps in our approach method solving BHFS-SDST. It is characterized by a shallow water wave propagation. These waves are forced to be braked and dissipated by frictional contact (see Table 11).

**Algorithm 5:** The WWO algorithm

```
λ , H_max , Π = {π^1, π^2, ..., π^{N_w}} and valuate the function value Z of
the population and determine π^best , π^worst
while the stopping criterion not satisfied do
    for p = 1 to N_w do
        π' ← π^p; (π^p given by DCP in propagation phase)
        if Z(π') < Z(π^p) then
            if Z(π') < Z(π^best) then
                π^best ← π' Applied PR for π' and update the
                value of Z(π') in breaking phase
            end
            π^p ← π' , H ← H_max
        else
            H ← H − 1
            if H == 0 then
                H ← H_max, π_p ← π' ;
                :Applied EP and choose the LS_1, LS_2 or GRASP
                in refraction phase
            else
                if Z(π') < Z(π^worst) then
                    π^worst ← π'
                else
                    if rand() ⩽ exp((Z(π^p)−Z(π'))/Temp) then
                        π^p ← π'
                    end
                end
            end
        end
    end
end
output: :π^best
```

In this approach the wave is considered as a solution or a sequence for our scheduling problem, characterized by two parameters: its wavelength $\lambda$ and its amplitude $H_{max}$. During its evolution a wave goes through three phases before stabilizing in its position of balance, the propagation phase, the refraction phase and the stopping or breaking phase. This model can be simulated in the optimization domain where the search space of the solutions represents the seabed. The propagation phase is characterized by the exploration of this space towards areas further away, looking for promising good solutions. In the refractive phase the search for a solution allows returning to other non-exploring neighborhood in the last phase. Finally, in the breaking phase the stabilized wave simulates the stabilization and convergence and the algorithm towards the best solution of the problem. Indeed, this good solution will be materialized by the stabilization of the wave towards its equilibrium position. Starting from this simulation phenomena, we adopt this model of WWO algorithm to solve our scheduling manufacturing minimization the total tardiness an earliness criterion in a BHFS-SDST (see Table 12).

**Table 12**
The used notation in DCP procedure.

| Notation | Description |
|---|---|
| $\pi$ | The current sequence in DCP algorithm |
| $\chi$ | The set of sub-sequence built by DCP algorithm |
| $\pi^{ig}$ | The sequence given by iterated greedy algorithm |
| $\lambda$ | The number of jobs extracted in destruction phase of DCP |

**Algorithm 6:** DCP

```
π = [π(1), π(2), ..., π(n)], Current sequence
χ ← ∅
% Destruction phase%
for h = 1 to λ do
    Extract a random job π(q) from the sequence π χ ← χ ∪ π(q),
    update the sequence π
end
% Construction phase%
π^ig ← π,%π^ig the iterated greedy sequence
for h = 1 to λ do
    Extract the job π(q) from the subset χ, where q ∈ [h, λ] insert the
    job π(q) on the different positions in the current sequence π^ig and
    choose the best position giving the smallest objective functional
    value Z, update χ
end
output: :π^ig
```

Our modification of the WWO algorithm aims at adopting the improvement procedure in the three main phases. The goal of this approach is to strengthen the algorithm by the PE for a good exploration of the solution search space. We describe below the modifications made as well as in the necessary parameters of each procedure in the different phases.

- Initialization phase: In this step a set of solutions is generated using the Nawaz–Enscore–Ham (NEH) algorithm (Nawaz et al., 1983) applied a set of solution forming the initial population $\mathcal{P} = \{\pi^1, \pi^2, \ldots, \pi^{N_w}\}$. Each solution is characterized by a wavelength $\lambda$ and a height $H$ of wave. To simplify the problem we will take the same value of the wavelength for all the solutions.
- Propagation phase: It is the main phase of exploration of the neighboring solutions space. A new solution is generated in the neighborhood by a defined enhancement procedure. In this phase we apply the destruction construction procedure (DCP) based on the IG algorithm. We vary the number of jobs $\lambda$ extracted from the sequence to simulate the real wave propagation for WWO. The Algorithm 6 describes the implementation steps of this procedure.
- Refraction phase: If during several generations there is no improvement of the best solution in the population, a refraction process is triggered and a new neighbor solution is created according to the neighborhood system to better explore all solutions. In

**Table 13**
The used notation in PR procedure.

| Notation | Description |
|---|---|
| $\pi$ | The current sequence in PR procedure |
| $\pi^{pr}$ | The path relinking sequence |
| $\kappa$ | The set of sequence built by PR procedure |
| $i_1, i_2$ | Index position of jobs in the current sequence |

this phase, we implement the EP approach with $LS_u$, $(u = 1, 2)$ with both versions and the $GRASP$ algorithm. Indeed, in this phase, the return to solutions that are not explored in the propagation phase is simulated. We propose here three versions of the $WWO_i$ $i = 1, 2, 3$ algorithm according to the chosen EP procedure. We specify that if a solution is not improved, we accept it with certain probability $p = exp^{\left(\frac{Z(\pi^p)-Z(\pi')}{Temp}\right)}$. This is implemented according to the simulated annealing model by the determination of the temperature $T_{emp}$ parameter; knowing that:

$$T_{emp} = T_0 \times \frac{\sum_{l=1}^{L}\sum_{j=1}^{n}(p_{jl} + s_{jjl})}{L \times n \times \sum_{l=1}^{L}(m_l)} \quad (2)$$

We adopt in this expression an adjustment coefficient $T_0$ depending on the size of the instance. This model is inspired by the model implemented in solving the permutation flow shop scheduling problem (Osman and Potts, 1989).

- Breaking phase: During the search for solutions in the neighborhood, if a new good solution is found ($Z(\pi') < Z(\pi^{best})$) after the propagation phase, the breaking procedure is started on a set of direction in order to generate a new solution for each direction. In this phase, we apply the algorithm of the path relinking (PR) technique (see Algorithm 7) to stabilize our algorithm and converge the meta-heuristic towards its good solution. In this procedure a solution $\pi$ is generated randomly, and a current solution $\pi^{best}$ is considered as a target. A set $\kappa$ of intermediate solutions are generated to reach the best solution. Each intermediate solution is evaluated. We retain the best solution $\pi^{pr}$ improving the solution $\pi^{best}$. If $\kappa$ is empty, the set of intermediate solutions is generated by the neighborhood system $\mathcal{N}_i$, $i = 1, \ldots, 4$. The technique makes it possible to retain the best solution $\pi^{pr}$ among those intermediate solutions which can be compared to $\pi^{best}$ (see Table 13).

**Algorithm 7:** PR

```
π = [π(1), π(2), ..., π(n)], Current sequence π^best
κ ← ∅, π^pr ← π % π^pr: sequence path relinking%
for i = 1 to n − 1 do
    if π(i) ≠ π^best(i) then
        Find the position k of the job π^best(i) in the sequence π and
        swap the job at position k with the job π(i) in the sequence π
        κ ← κ ∪ π
    end
end
if κ ≠ ∅ then
    Select the sequence π^pr giving the smallest value of Z in κ
else
    for i = 1 to 4 do
        Generate two random positions i_1, i_2, i_1 ≠ i_2 apply the
        procedure N_i for π(i_1) and π(i_2) given π.
        κ ← κ ∪ π
    end
    Select the sequence π^pr giving the smallest value of Z in κ
end
output: :π^pr
```

### 4.4. Initialization algorithms

The initialization phase is an important step using the metaheuristic approaches. Indeed, when starting with a good solution, this allows saving considerable computation time. Our various proposed algorithms are population metaheuristics whose parameters vary for each category of algorithm. We propose a constructive procedure called in the scheduling literature NEH algorithm, applied to the PFSP to generate the initial population. We suggest an adopted version of the NEH algorithm to solve the BHFS-SDST problem. We propose different expressions by calculating a set of $\sigma_j^i, i = 1, \ldots, 4$ including the parameters of instance. For each expression, we classify the jobs in descending order and we apply the NEH procedure. In case the population size is not reached, we complete the missing remainder with randomized generated solutions. For this, we compute a set of expressions by determining the parameter $\sigma_j^i$, knowing that

- $\sigma_j^1 = \xi \times \sum_{l=1}^{L} p_{jl} + (1-\xi) \times (rj + d_j)$
- $\sigma_j^2 = \xi \times \sum_{l=1}^{L} p_{jl} + (1-\xi) \times \sum_{l=1}^{L}\sum_{k=1}^{n} s_{jkl}$
- $\sigma_j^3 = \xi \times (r_j + d_j) + (1-\xi) \times \sum_{l=1}^{L}\sum_{k=1}^{n}(s_{kjl})$
- $\sigma_j^4 = \xi \times (r_j + |d_j - \sum_{l=1}^{L} p_{jl}|) + (1-\xi) \times \sum_{l=1}^{L}\sum_{k=1}^{n} s_{kjl}$

In the expressions of $\sigma_j^i$, the $\xi \in [0, 1]$, we highlight the processing time $p_{jl}$, the setup time $s_{kjl}$, the earliest time $r_j$ as well as the due time $d_j$ of each job. We adopt a weighting coefficient between the two terms of each expression of $\sigma_j^i$. This variation is made in the interval $[0, 1]$ to take into consideration the effect of each term in the ranking obtained in ascending order. This order is necessary to better apply the NEH algorithm by simulating the scheduling of jobs as if they were executed in a conventional flow shop with $L$ machines.

By summarizing the proposed resolution approaches to solve our problem, we find that we have six MBO and WWO based algorithms. For the first approach we distinguish three algorithms $EMBO_i$, $i = 1, 2, 3$. For the second approach we have three algorithms $WWO_i$, $i = 1, 2, 3$. The six versions of algorithms differ according to the proposed improvement procedure. Our vision is to offer manufacturers a simulation model that can be adapted to several cases of machine configurations. In particular, our model can also fit $HFS((PM)_l)_{l=1}^{L}|r_j, d_j, SDST|\sum_{j=1}^{n}(E_j + T_j)$, when each machine has the same speed of production. Indeed, in this description, we will highlight a software platform to offer business managers a great flexibility of adaptation and decision-making. The purpose of the model is to increase the rationalization of production resources and to satisfy the constraints imposed by customers.

## 5. Computing results

### 5.1. Instance settings

The verification of the effectiveness of the proposed metaheuristic algorithms for solving the BHFS-SDST problem is established by a study of numerical simulations. Validation tests are performed on instances generated randomly involving the number of jobs, the number of machines per stage and the number of stages. We distinguish two main instance types: the type 1 and the type 2. This classification is made according to the size of the simulated instance. In each type of instance, we identify the number of machines per stage, the number of stages and the number of jobs in the production batch.

- **type 1**: For this type, we group small and medium size instances, such as $n \in \{10, 20, \ldots, 90\}$, the number of stages is $L \in \{2, 3, \ldots, 6\}$ and the machine number per stage is $m_l \in \{2, 3, \ldots, 8\}$. We note that for the first type of instances, the processing times per stage are randomly generated according to a uniform law in the interval $[10, 49]$ and the setup times are also generated in a random manner according to a uniform law in the interval $[1, 10]$.

**Table 14**
Level factor for $EMBO_i$.

| Algorithm | Parameter | Level factor | | |
|---|---|---|---|---|
| | | 1 | 2 | 3 |
| $MBO+LS_1$ : $EMBO_1$ | $\mu_1$ | $n/3$ | $n/2$ | $2n/3$ |
| $MBO+LS_2$ : $EMBO_2$ | $\mu_2$ | $n/3$ | $n/2$ | $2n/3$ |
| MBO+GRASP : $EMBO_3$ | $\alpha$ | 0.5 | 0.7 | 0.9 |

**Table 15**
Level factor for $WWO_i$.

| Algorithm | Parameter | Level factor | | |
|---|---|---|---|---|
| | | 1 | 2 | 3 |
| $WWO+DCP+PRP+LS_1$ : $WWO_1$ | $\mu_1$ | $n/3$ | $n/2$ | $2n/3$ |
| $WWO+DCP+PRP+LS_2$ : $WWO_2$ | $\mu_2$ | $n/3$ | $n/2$ | $2n/3$ |
| WWO+DCP+PRP+GRASP : $WWO_3$ | $\alpha$ | 0.5 | 0.7 | 0.9 |

- **type 2**: The characteristics of instances of type 2 are also defined by specifying $n, L, m_l$, knowing that $n \in \{100, 120, \ldots, 220\}$, the number of stages $L \in \{6, 7, \ldots, 10\}$ and the number of machines per stage is $m_l \in \{6, 7, \ldots, 12\}$. This type of instance is qualified as relatively larger instances, depending on the number of stages, when the parameters $n, L$ and $m_l$ are bigger. For this type , it is considered that the processing times are generated according to a uniform law in the interval $[50, 99]$ and the setup times are generated also in a random manner according to a uniform law in the interval $[10, 20]$.

For both types of instances, we have chosen that the setup times of the jobs do not exceed 20% the operating times which generally represent the frequently encountered industrial cases. We note that the speeds of the machines are defined by $v_{il} \in \{1, \ldots, 5\}$. Earliest arrival dates are also randomly generated, such as $r_j \in [1, 30]$. Besides, the due date $d_j$ will be calculated according to the processing times and the setup times. We have chosen that $dj$ will be computed by the expression of Eq. (3).

$$d_j = \zeta \times (r_j + \sum_{l=1}^{L}(p_{jl} + \sum_{h=1}^{n}(\frac{s_{hjl}}{m_l}))) \tag{3}$$

In this expression, we introduce the parameters of each job. In addition, the coefficient is chosen according to the type of instance. We took $\zeta \in \{1, 2, 3\}$ for type 1 and $\zeta \in \{4, 5, 6\}$ for type 2. For each meta-heuristics one chooses the necessary parameters according to the size of the problem defined previously to better check its effectiveness. We have defined a set of levels for the six algorithms. For the algorithms $EMBO_i$, $i = 1, 2, 3$ we give the parameters of settings. We consider that $\eta \in \{31, 51, 71\}$. Likewise we consider that $\omega \in \{20, 40, 60\}$. In Table 14, the parameters are given to illustrate neighborhood size exploration for different versions of $EMBO_i$

We do the same thing for the setting of the parameters of the implementation of our algorithms $WWO_i$, $i = 1, 2, 3$. In this approach, we propose to do the analysis for $N_w \in \{15, 25, 35\}$, with $H_{max} \in [2, 6]$. We also note that the wavelength will also be chosen based on the number of jobs processed, such as $\lambda \in \{n/3, n/2, 2n/3\}$. The Table 15 represents the level factor tested for the different algorithm $WWO_i$.

The analysis and the comparative study between the various proposed algorithms are carried out on a set of instance covering the different sizes of the problems. The first comparative study is made in terms of the quality of the solution obtained. We propose comparing our algorithms by calculating the average relative of the percentage deviation $\overline{RPD}$. The Eq. (4) allows giving the expression of $\overline{RPD}$. We consider ten simulated instances that we consider largely sufficient to make judgments on the quality of the result for each algorithm.

$$\overline{RPD} = \frac{1}{10} \times \sum_{r=1}^{10}\left(\frac{Z^r_{algo} - Z_{best}}{Z_{best}}\right) \times 100\% \tag{4}$$

where $Z^r_{algo}$ is the solution generated by a specific algorithm in the $r$th replication for a given instance. $Z_{best}$ represents the minimum value of the objective function for all algorithms. To limit computation, we consider that $\tau = \rho \times n \times TM$ in milliseconds, where $TM = \sum_{l=1}^{L} m_l$ is total machines. We propose to evaluate the results for the values of $\rho \in \{10, 20, 30\}$.

In Fig. 9, we represent a comparative study between the three versions of the MBO and WWO algorithms. This study concerns the relatively small and medium sized problems classified in type 1 of instances.

Our goal is to show the impact of the size of the neighborhood system for the various algorithms on the quality of the obtained solution. This is defined in the local search improvement procedures $LS_u$,$(u = 1, 2)$ where we consider the two parameters $\mu_1$ and $\mu_2$ for EP and the parameter $\alpha$ for GRASP procedure. For the three types of improvement, we consider three experimental levels for each neighborhood parameter. The results are given for different values of jobs number $n$ and for a number of machines per stage respecting the proposed classification. We note that when the two parameters $\mu_1$ and $\mu_2$ are in the level 3, this allows saving the best results for the algorithms $EMBO_i$, (i = 1,2) and $WWO_i$, (i = 1,2). When the $\alpha$ parameter is at level 1 it allows recording the best performance for the $EMBO_3$ and $WWO_3$ algorithms.

In Fig. 10, we also represent the same analysis concerning the effect of the size of the neighborhood system for the instances considered with large size. For this study, we vary the three parameters $\mu_1$, $\mu_2$ and $\alpha$ according to the three levels as defined above. We have chosen representative instances of the second type 2 family, highlighting a significant large number of jobs and machines. The same observation is recorded for the proposed algorithms. Indeed, when $\mu_1$, $\mu_2$ are in level 3 the $EMBO_i$, (i = 1,2) and $WWO_i$, (i= 1,2) algorithms given the best solutions compared at other levels. Also, when $\alpha$ is at the 1 level, the $EMBO_3$ and $WWO_3$ algorithms perform well compared to the cases of $\alpha$ when it is at the levels 2 and 3. In conclusion, in the study for the two cases of simulated instances type, we note that when the neighborhood system is enlarged it allows better exploration of the neighborhood of the current solution. For the six algorithms, the best performance in terms of the quality of the solution obtained is given for a large neighborhood system.

We continue the comparative study between our algorithms to verify their effectiveness. Thereafter, we propose a set of six tables, two for each type of instance. We propose this bench-marking studies by making an average of ten replications for each instance according to the formula given by Eq. (3). For the two categories of problems, we tested our algorithms on a set of 150 instances. We limit ourselves to the computing time given by $tau$ by varying $\rho$ for each type of problem. In Tables 16–18, we represent the result for $\rho = 10$, $\rho = 20$ and $\rho = 30$ for type 1 instances. The results found are from the simulation on 81 instances. The best value of $\overline{RPD}$ is recorded by the algorithm $WWO_1$ for the two values of $\rho$, such as 0.17%, 0.25% and 0.09% for $\rho = 10$, $\rho = 20$ and $\rho = 30$ respectively. We find that the $WWO_1$ algorithm gives a good solution and records a success rate of 77.77% for all type 1 instances. In the same context, Tables 19–21 represent the results of all type 2 of instances. The analysis is done as before, considering the same values of $\rho = 10$, $\rho = 20$ and $\rho = 30$. The total number of problems tested in this category is 63 instances. The approach remains identical but the adjustment parameters for the algorithms differ from those used in type 1 instances. Indeed, the population size of the algorithms as well as the size of the neighborhood systems will be enlarged according to the size of the instance. The results show that the algorithm $WWO_1$ remains the best giving minimum values for the three cases of $\rho = 10$, $\rho = 20$ and $\rho = 30$ of 0.00%.

We continue our analysis of the performance of the proposed algorithms. In Figs. 11–13, we represent the box plots of the different algorithms for a set of test problems representative of type 1 of instances. The simulation tests concern the parameter $\rho \in \{10, 20, 30\}$ of the limit computation time $\tau$, for the highest population size numbers

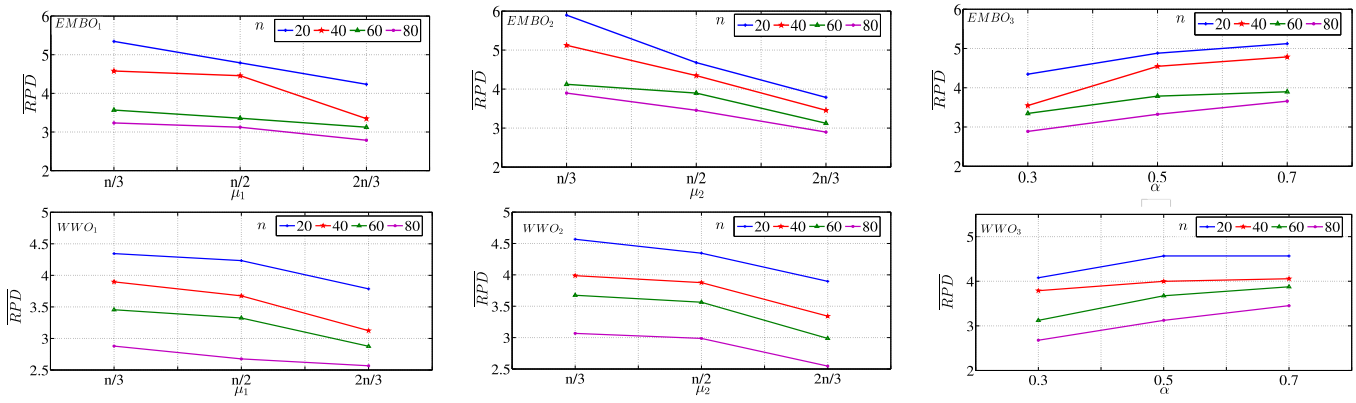



**Fig. 9.** Interaction between level factor of algorithms and the number of jobs for type 1 instances.

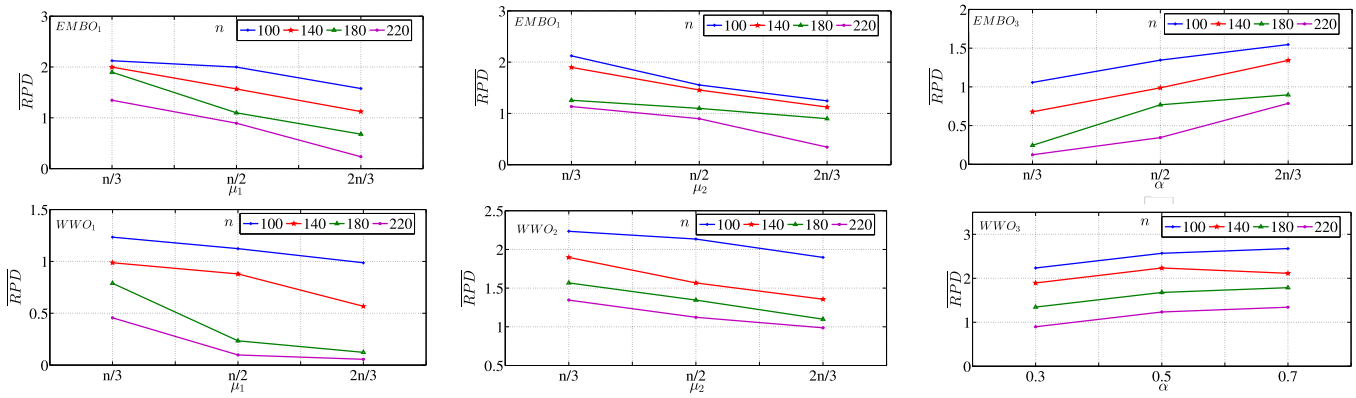



**Fig. 10.** Interaction between level factor of algorithms and the number of jobs for type 2 instances.

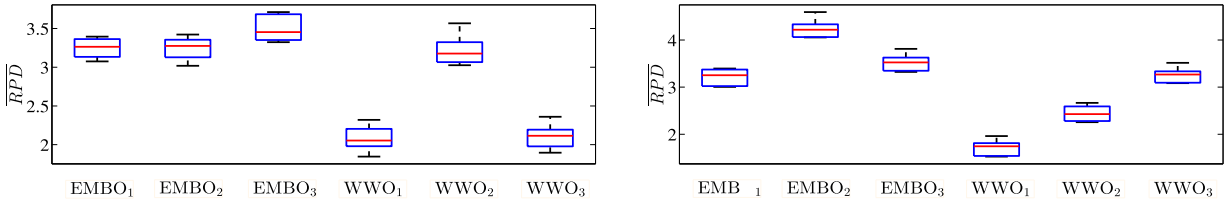



**Fig. 11.** The box plot for instance type 1 with $\rho = 10$ : $10 \times 4 \times 10$ (left), $20 \times 4 \times 10$ (right).

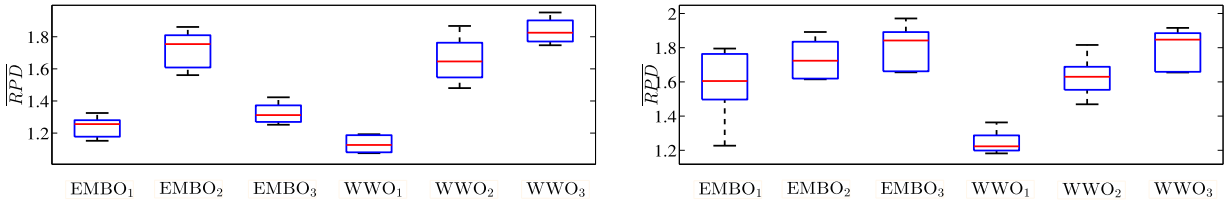



**Fig. 12.** The box plot for instance type 1 with $\rho = 20$ : $30 \times 5 \times 14$ (left), $50 \times 6 \times 16$ (right).

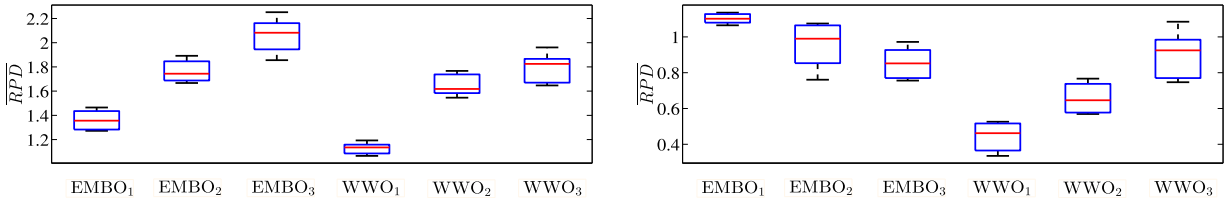



**Fig. 13.** The box plot for instance type 1 with $\rho = 30$ : $80 \times 6 \times 16$ (left), $90 \times 6 \times 20$ (right).

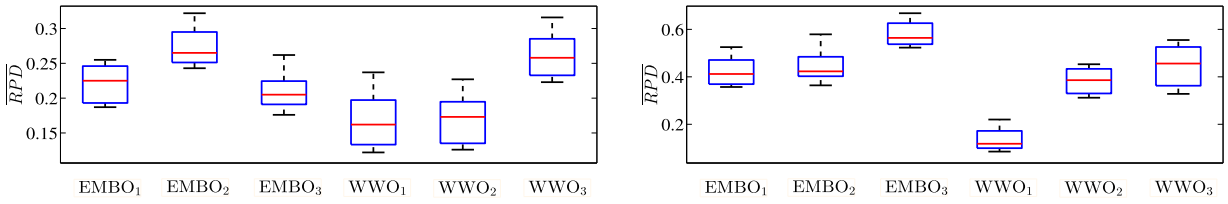



**Fig. 14.** The box plot for instance type 2 with $\rho = 10$ : 140 × 8 × 24 (left), 160 × 9 × 28 (right).

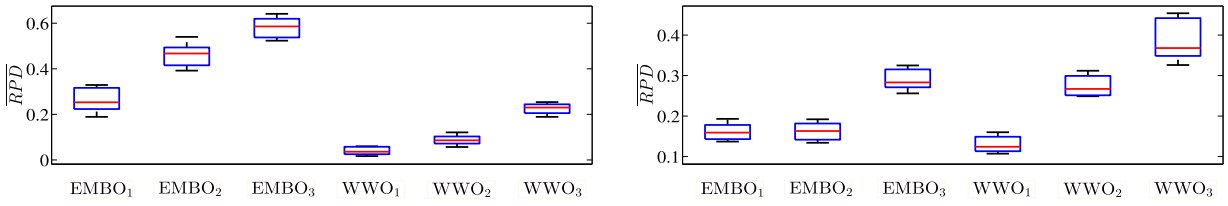



**Fig. 15.** The box plot for instance type 2 with $\rho = 20$ : 140 × 8 × 24 (left), 180 × 10 × 25 (right).

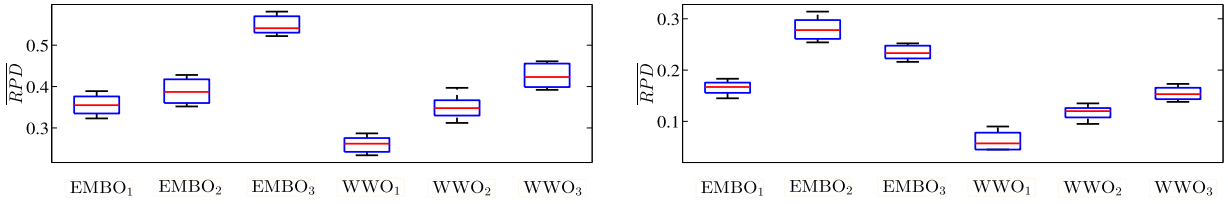



**Fig. 16.** The box plot for instance type 2 with $\rho = 30$ : 140 × 8 × 24 (left), 180 × 10 × 20 (right).

**Table 16**
$\overline{RPD}$ **-type 1**- $\rho = 10$. The best results are in bold.

| Instance $n \times L \times TM$ | $EMBO_1$ | $EMBO_2$ | $EMBO_3$ | $WWO_1$ | $WWO_2$ | $WWO_3$ |
|---|---|---|---|---|---|---|
| 10 × 4 × 8 | 2.32 | 3.23 | 4.56 | **2.25** | 3.21 | 3.23 |
| 10 × 4 × 10 | 3.22 | 3.28 | 3.56 | **1.95** | 3.21 | 2.13 |
| 10 × 4 × 12 | 3.45 | 4.43 | 5.26 | **3.28** | 4.21 | 4.43 |
| 20 × 4 × 10 | 3.12 | 4.23 | 3.56 | **1.75** | 2.41 | 3.24 |
| 20 × 4 × 12 | 4.22 | 5.33 | 4.76 | **3.85** | 4.44 | 5.54 |
| 20 × 4 × 14 | 3.92 | 4.17 | 4.26 | 3.85 | **2.82** | 3.74 |
| 30 × 5 × 12 | 1.32 | 2.43 | 2.58 | **0.85** | 1.35 | 2.64 |
| 30 × 5 × 14 | 2.82 | 3.33 | 3.16 | **2.15** | 2.57 | 2.84 |
| 30 × 5 × 16 | 3.22 | 4.23 | 4.36 | 2.73 | **1.87** | 2.64 |
| 40 × 6 × 14 | 6.22 | 6.43 | 5.56 | **4.14** | 5.23 | 4.39 |
| 40 × 6 × 16 | 5.33 | 5.73 | 4.86 | **3.23** | 4.25 | 4.86 |
| 40 × 6 × 18 | 2.32 | 3.53 | 3.86 | 2.75 | **1.71** | 2.42 |
| 50 × 6 × 16 | 3.82 | 3.23 | 4.56 | **2.52** | 3.41 | 4.21 |
| 50 × 6 × 18 | 1.24 | 2.63 | 2.76 | 1.25 | **1.11** | 2.31 |
| 50 × 6 × 20 | 2.75 | 3.33 | 2.68 | **2.25** | 3.31 | 3.54 |
| 60 × 6 × 16 | 1.25 | 2.33 | 1.67 | **0.95** | 2.31 | 1.46 |
| 60 × 6 × 20 | 1.74 | 2.23 | 1.64 | **0.75** | 1.51 | 1.37 |
| 60 × 6 × 24 | 1.37 | 2.14 | 2.18 | **1.17** | 1.91 | 1.94 |
| 70 × 6 × 16 | 1.52 | 1.23 | 1.29 | **0.88** | 1.13 | 1.38 |
| 70 × 6 × 20 | 1.19 | 1.23 | 1.25 | 0.97 | **0.87** | 1.05 |
| 70 × 6 × 24 | 1.75 | 1.63 | 1.95 | **1.25** | 1.31 | 1.64 |
| 80 × 6 × 16 | 1.18 | 1.63 | 2.56 | **0.65** | 1.41 | 1.24 |
| 80 × 6 × 20 | 1.42 | 1.53 | 2.26 | **0.92** | 1.81 | 1.34 |
| 80 × 6 × 24 | 0.92 | 0.83 | 0.96 | **0.45** | 0.81 | 0.94 |
| 90 × 6 × 16 | 1.25 | 1.35 | 1.86 | **1.05** | 1.23 | 1.42 |
| 90 × 6 × 20 | 0.49 | 0.63 | 0.76 | 0.25 | **0.21** | 0.14 |
| 90 × 6 × 24 | 0.89 | 0.94 | 0.86 | **0.17** | 0.27 | 0.28 |
| Average | 2.38 | 2.86 | 2.94 | 1.78 | 2.21 | 2.39 |

**Table 17**
$\overline{RPD}$ **-type 1**- $\rho = 20$. The best results are in bold.

| Instance $n \times L \times TM$ | $EMBO_1$ | $EMBO_2$ | $EMBO_3$ | $WWO_1$ | $WWO_2$ | $WWO_3$ |
|---|---|---|---|---|---|---|
| 10 × 4 × 8 | 3.22 | 4.28 | 4.22 | **2.54** | 3.21 | 3.89 |
| 10 × 4 × 10 | 2.28 | 2.56 | 2.98 | 2.74 | **2.45** | 2.31 |
| 10 × 4 × 12 | 1.45 | 2.67 | 1.86 | **1.34** | 1.86 | 1.56 |
| 20 × 4 × 10 | 2.23 | 2.86 | 2.43 | **1.86** | 2.45 | 1.94 |
| 20 × 4 × 12 | 1.34 | 1.34 | 1.56 | **1.11** | 1.76 | 1.89 |
| 20 × 4 × 14 | 0.57 | 0.56 | 0.65 | **0.34** | 0.67 | 0.98 |
| 30 × 5 × 12 | 0.45 | 0.56 | 0.68 | **0.23** | 0.65 | 0.55 |
| 30 × 5 × 14 | 1.23 | 1.75 | 1.34 | **1.12** | 1.67 | 1.89 |
| 30 × 5 × 16 | 0.39 | 0.64 | 0.76 | **0.45** | 0.89 | 0.54 |
| 40 × 6 × 14 | 1.56 | 1.63 | 1.76 | 1.85 | **1.36** | 1.82 |
| 40 × 6 × 16 | 1.94 | 2.02 | 2.78 | 1.83 | **1.56** | 1.19 |
| 40 × 6 × 18 | 0.68 | 0.94 | 0.98 | **0.57** | 0.87 | 0.88 |
| 50 × 6 × 16 | 1.65 | 1.68 | 1.86 | **1.22** | 1.65 | 1.87 |
| 50 × 6 × 18 | 1.08 | 1.17 | 1.24 | **0.67** | 0.78 | 1.05 |
| 50 × 6 × 20 | 0.67 | 0.45 | 0.82 | **0.29** | 0.34 | 0.45 |
| 60 × 6 × 16 | 3.65 | 3.73 | 4.08 | 3.45 | **3.31** | 3.54 |
| 60 × 6 × 20 | 3.75 | 4.03 | 4.18 | **2.25** | 3.61 | 3.74 |
| 60 × 6 × 24 | 2.25 | 2.83 | 2.72 | **1.75** | 2.37 | 2.67 |
| 70 × 6 × 16 | 1.15 | 1.23 | 1.88 | **1.05** | 1.11 | 1.14 |
| 70 × 6 × 20 | 1.17 | 2.03 | 2.08 | **1.09** | 1.31 | 1.14 |
| 70 × 6 × 24 | 2.65 | 2.83 | 3.08 | 2.25 | **1.11** | 1.67 |
| 80 × 6 × 16 | 1.18 | 1.63 | 2.56 | **0.75** | 1.41 | 1.64 |
| 80 × 6 × 20 | 1.42 | 1.53 | 2.06 | **0.92** | 1.81 | 1.94 |
| 80 × 6 × 24 | 0.85 | 0.88 | 1.06 | **0.45** | 0.81 | 0.87 |
| 90 × 6 × 16 | 1.25 | 1.35 | 1.86 | 1.05 | **1.03** | 1.42 |
| 90 × 6 × 20 | 1.09 | 1.63 | 2.06 | **0.25** | 0.29 | 0.44 |
| 90 × 6 × 24 | 1.49 | 1.63 | 1.76 | **0.45** | 0.81 | 0.96 |
| Average | 1.57 | 1.86 | 2.05 | 1.21 | 1.52 | 1.62 |

for the six versions of algorithms. The plots show that the $WWO_1$ algorithm records the best result among the other algorithms.

In Figs. 14–16, we represent the comparative study between the different algorithms for the test problems in type 2 of instances. We consider the same conditions and hypotheses for type 1 of instances. We consider that $\rho \in \{10, 20, 30\}$ for the time limit $\tau$ to calculate the value of $\overline{RPD}$. We choose that the population size of the algorithms are the largest values. Indeed, for the $EMBO_i$, $i = 1, 2, 3$ algorithms, the

**Table 18**
$\overline{RPD}$ -**type 1**- $\rho = 30$. The best results are in bold.

| Instance $n \times L \times TM$ | $EMBO_1$ | $EMBO_2$ | $EMBO_3$ | $WWO_1$ | $WWO_2$ | $WWO_3$ |
|---|---|---|---|---|---|---|
| $10 \times 4 \times 8$ | 4.87 | 4.65 | 5.12 | **3.23** | 3.85 | 3.98 |
| $10 \times 4 \times 10$ | 4.11 | 4.55 | 4.67 | **3.35** | 3.79 | 4.01 |
| $10 \times 4 \times 12$ | 3.13 | 3.45 | 3.19 | **2.12** | 2.79 | 2.83 |
| $20 \times 4 \times 10$ | 2.82 | 3.33 | 3.16 | 2.75 | **2.41** | 3.24 |
| $20 \times 4 \times 12$ | 3.35 | 3.65 | 4.18 | **2.15** | 3.84 | 3.64 |
| $20 \times 4 \times 14$ | 2.78 | 2.56 | 2.86 | 2.95 | **2.45** | 3.84 |
| $30 \times 5 \times 12$ | 1.37 | 2.65 | 1.87 | **1.07** | 1.17 | 1.85 |
| $30 \times 5 \times 14$ | 2.23 | 2.33 | 2.83 | **1.18** | 1.64 | 2.06 |
| $30 \times 5 \times 16$ | 1.15 | 1.69 | 1.82 | **0.86** | 1.04 | 1.13 |
| $40 \times 6 \times 14$ | 2.85 | 2.36 | 2.89 | 2.86 | **1.23** | 2.39 |
| $40 \times 6 \times 16$ | 1.13 | 1.83 | 1.94 | **0.67** | 1.09 | 1.16 |
| $40 \times 6 \times 18$ | 0.89 | 1.03 | 1.06 | **0.15** | 0.22 | 0.65 |
| $50 \times 6 \times 16$ | 1.12 | 1.73 | 1.86 | **1.02** | 1.11 | 1.12 |
| $50 \times 6 \times 18$ | 2.94 | 3.83 | 3.16 | **2.25** | 2.86 | 3.86 |
| $50 \times 6 \times 20$ | 3.68 | 4.75 | 3.88 | 3.75 | **3.64** | 3.77 |
| $60 \times 6 \times 16$ | 1.05 | 1.63 | 1.17 | **0.85** | 1.01 | 1.21 |
| $60 \times 6 \times 20$ | 1.06 | 1.14 | 1.56 | **0.62** | 1.02 | 1.05 |
| $60 \times 6 \times 24$ | 1.77 | 2.14 | 2.05 | **1.47** | 1.62 | 1.74 |
| $70 \times 6 \times 16$ | 1.02 | 1.13 | 1.23 | 0.62 | **0.23** | 0.87 |
| $70 \times 6 \times 20$ | 3.62 | 3.75 | 3.86 | **2.77** | 2.85 | 2.92 |
| $70 \times 6 \times 24$ | 1.86 | 2.03 | 2.05 | **1.05** | 1.17 | 1.84 |
| $80 \times 6 \times 16$ | 1.37 | 1.83 | 2.07 | **1.15** | 1.62 | 1.84 |
| $80 \times 6 \times 20$ | 1.72 | 1.77 | 1.85 | **0.67** | 1.65 | 1.68 |
| $80 \times 6 \times 24$ | 1.02 | 1.13 | 1.05 | **0.15** | 0.62 | 0.88 |
| $90 \times 6 \times 16$ | 1.67 | 2.12 | 2.06 | 1.15 | **1.03** | 1.65 |
| $90 \times 6 \times 20$ | 1.09 | 0.93 | 0.86 | **0.45** | 0.67 | 0.85 |
| $90 \times 6 \times 24$ | 0.77 | 0.98 | 1.06 | **0.09** | 0.23 | 0.33 |
| Average | 2.09 | 2.62 | 2.68 | 1.53 | 1.73 | 2.08 |

**Table 19**
$\overline{RPD}$ -**type 2**- $\rho = 10$. The best results are in bold.

| Instance $n \times L \times TM$ | $EMBO_1$ | $EMBO_2$ | $EMBO_3$ | $WWO_1$ | $WWO_2$ | $WWO_3$ |
|---|---|---|---|---|---|---|
| $100 \times 7 \times 20$ | 0.78 | 0.76 | 0.68 | **0.34** | 0.64 | 0.89 |
| $100 \times 7 \times 24$ | 0.14 | 0.34 | 0.16 | **0.08** | 0.09 | 0.13 |
| $100 \times 7 \times 28$ | 0.22 | 0.24 | 0.33 | 0.19 | **0.17** | 0.21 |
| $120 \times 7 \times 25$ | 0.28 | 0.39 | 0.45 | 0.21 | **0.12** | 0.27 |
| $120 \times 7 \times 30$ | 0.32 | 0.35 | 0.43 | **0.15** | 0.24 | 0.22 |
| $120 \times 7 \times 35$ | 0.24 | 0.26 | 0.31 | **0.02** | 0.08 | 0.09 |
| $140 \times 8 \times 24$ | 0.09 | 0.16 | 0.16 | **0.04** | 0.07 | 0.08 |
| $140 \times 8 \times 28$ | 0.22 | 0.28 | 0.24 | **0.15** | 0.17 | 0.26 |
| $140 \times 8 \times 32$ | 0.16 | 0.26 | 0.38 | **0.04** | 0.17 | 0.28 |
| $160 \times 9 \times 20$ | 0.25 | 0.27 | 0.32 | 0.17 | **0.14** | 0.24 |
| $160 \times 9 \times 24$ | 0.27 | 0.28 | 0.49 | **0.17** | 0.26 | 0.28 |
| $160 \times 9 \times 28$ | 0.41 | 0.43 | 0.57 | **0.11** | 0.38 | 0.47 |
| $180 \times 10 \times 20$ | 0.19 | 0.11 | 0.29 | **0.08** | 0.09 | 0.12 |
| $180 \times 10 \times 25$ | 1.23 | 1.37 | 1.48 | 1.21 | **1.16** | 1.27 |
| $180 \times 10 \times 30$ | 0.09 | 0.08 | 0.07 | 0.05 | **0.04** | 0.09 |
| $200 \times 10 \times 20$ | 0.41 | 0.52 | 0.76 | **0.05** | 0.29 | 1.15 |
| $200 \times 10 \times 25$ | 0.16 | 0.18 | 0.29 | **0.01** | 0.05 | 0.06 |
| $200 \times 10 \times 30$ | 0.13 | 0.22 | 0.24 | **0.09** | 0.12 | 0.17 |
| $220 \times 10 \times 20$ | 0.05 | 0.08 | 0.09 | **0.00** | 0.02 | 0.01 |
| $220 \times 10 \times 25$ | 0.04 | 0.07 | 0.07 | **0.01** | 0.04 | 0.07 |
| $220 \times 10 \times 30$ | 0.05 | 0.06 | 0.07 | **0.00** | 0.02 | 0.02 |
| Average | 0.27 | 0.31 | 0.37 | 0.15 | 0.20 | 0.30 |

size is 71 and for the $WWO_i$, $i = 1, 2, 3$ algorithms, the population size is 35. The results shows that the $WWO_1$ records the best performance compared to the others algorithms.

## 6. Conclusions

The hybrid flow shop scheduling problem is very interesting in engineering optimization and it is an attractive model in operation research. In this paper, we have proposed two nature inspired metaheuristics to solve blocking hybrid flow shop scheduling problem. For each stage of the manufacturing system, we have considered uniform parallel machines under the constraint of sequence dependent setup time. The objective is to minimize the total tardiness and earliness of all jobs

**Table 20**
$\overline{RPD}$ -**type 2**- $\rho = 20$. The best results are in bold.

| Instance $n \times L \times TM$ | $EMBO_1$ | $EMBO_2$ | $EMBO_3$ | $WWO_1$ | $WWO_2$ | $WWO_3$ |
|---|---|---|---|---|---|---|
| $100 \times 7 \times 20$ | 2.14 | 2.24 | 2.26 | **1.24** | 1.34 | 1.67 |
| $100 \times 7 \times 24$ | 1.28 | 1.37 | 1.54 | **1.12** | 1.18 | 1.36 |
| $100 \times 7 \times 28$ | 1.82 | 1.93 | 1.97 | 1.43 | **1.39** | 1.88 |
| $120 \times 7 \times 25$ | 0.68 | 0.87 | 0.45 | **0.25** | 0.45 | 1.19 |
| $120 \times 7 \times 30$ | 1.03 | 1.08 | 1.27 | **0.29** | 0.87 | 0.96 |
| $120 \times 7 \times 35$ | 0.44 | 0.67 | 0.87 | **0.17** | 0.65 | 0.83 |
| $140 \times 8 \times 24$ | 0.37 | 0.48 | 0.59 | **0.03** | 0.08 | 0.21 |
| $140 \times 8 \times 28$ | 0.18 | 0.24 | 0.45 | 0.12 | **0.08** | 0.14 |
| $140 \times 8 \times 32$ | 0.25 | 0.27 | 0.36 | **0.03** | 0.16 | 0.18 |
| $160 \times 9 \times 20$ | 0.26 | 0.28 | 0.47 | **0.05** | 0.18 | 0.28 |
| $160 \times 9 \times 24$ | 0.06 | 0.04 | 0.01 | **0.00** | 0.02 | 0.09 |
| $160 \times 9 \times 28$ | 0.24 | 0.27 | 0.33 | **0.17** | 0.21 | 0.28 |
| $180 \times 10 \times 20$ | 0.27 | 0.23 | 0.48 | 0.17 | **0.11** | 0.16 |
| $180 \times 10 \times 25$ | 0.17 | 0.18 | 0.28 | **0.13** | 0.24 | 0.39 |
| $180 \times 10 \times 30$ | 0.31 | 0.42 | 0.45 | 0.23 | **0.21** | 0.43 |
| $200 \times 10 \times 20$ | 0.18 | 0.27 | 0.35 | 0.08 | **0.05** | 0.18 |
| $200 \times 10 \times 25$ | 0.06 | 0.08 | 0.09 | **0.00** | 0.01 | 0.05 |
| $200 \times 10 \times 30$ | 0.09 | 0.11 | 0.12 | **0.02** | 0.04 | 0.05 |
| $220 \times 10 \times 20$ | 0.08 | 0.07 | 0.09 | **0.00** | 0.01 | 0.05 |
| $220 \times 10 \times 25$ | 0.14 | 0.26 | 0.21 | **0.00** | 0.07 | 0.12 |
| $220 \times 10 \times 30$ | 0.13 | 0.16 | 0.15 | **0.00** | 0.08 | 0.13 |
| Average | 0.48 | 0.54 | 0.60 | 0.26 | 0.35 | 0.50 |

**Table 21**
$\overline{RPD}$ -**type 2**- $\rho = 30$. The best results are in bold.

| Instance $n \times L \times TM$ | $EMBO_1$ | $EMBO_2$ | $EMBO_3$ | $WWO_1$ | $WWO_2$ | $WWO_3$ |
|---|---|---|---|---|---|---|
| $100 \times 7 \times 20$ | 1.65 | 1.87 | 1.88 | **1.22** | 1.74 | 1.77 |
| $100 \times 7 \times 24$ | 1.24 | 1.45 | 1.26 | **1.12** | 1.18 | 1.25 |
| $100 \times 7 \times 28$ | 0.88 | 0.94 | 1.05 | **0.73** | 0.87 | 0.93 |
| $120 \times 7 \times 25$ | 0.37 | 0.69 | 0.65 | **0.25** | 0.32 | 0.47 |
| $120 \times 7 \times 30$ | 0.22 | 0.32 | 0.45 | 0.16 | **0.15** | 0.19 |
| $120 \times 7 \times 35$ | 0.15 | 0.18 | 0.23 | **0.07** | 0.11 | 0.12 |
| $140 \times 8 \times 24$ | 0.18 | 0.24 | 0.23 | 0.14 | **0.09** | 0.17 |
| $140 \times 8 \times 28$ | 0.36 | 0.38 | 0.54 | **0.26** | 0.38 | 0.42 |
| $140 \times 8 \times 32$ | 0.15 | 0.36 | 0.42 | **0.07** | 0.15 | 0.31 |
| $160 \times 9 \times 20$ | 0.38 | 0.47 | 0.53 | **0.23** | 0.34 | 0.34 |
| $160 \times 9 \times 24$ | 0.27 | 0.33 | 0.38 | **0.12** | 0.22 | 0.24 |
| $160 \times 9 \times 28$ | 0.48 | 0.54 | 0.67 | 0.32 | **0.25** | 0.42 |
| $180 \times 10 \times 20$ | 0.17 | 0.28 | 0.22 | **0.07** | 0.11 | 0.15 |
| $180 \times 10 \times 25$ | 1.31 | 1.33 | 1.42 | **1.11** | 1.21 | 1.28 |
| $180 \times 10 \times 30$ | 0.12 | 0.23 | 0.15 | 0.08 | **0.07** | 0.11 |
| $200 \times 10 \times 20$ | 0.52 | 0.64 | 0.86 | **0.11** | 0.22 | 1.07 |
| $200 \times 10 \times 25$ | 0.21 | 0.28 | 0.32 | 0.09 | **0.07** | 0.12 |
| $200 \times 10 \times 30$ | 0.34 | 0.37 | 0.48 | **0.13** | 0.25 | 0.32 |
| $220 \times 10 \times 20$ | 0.15 | 0.18 | 0.19 | **0.07** | 0.13 | 0.17 |
| $220 \times 10 \times 25$ | 0.07 | 0.08 | 0.12 | **0.00** | 0.06 | 0.09 |
| $220 \times 10 \times 30$ | 0.08 | 0.09 | 0.11 | **0.03** | 0.07 | 0.09 |
| Average | 0.44 | 0.53 | 0.57 | 0.30 | 0.38 | 0.47 |

subject to the constraint of realize and due dates. We have suggested efficient algorithms based on the migratory bird optimization algorithm and the water wave optimization algorithm. These new approaches are considered to be hybrid algorithms that integrate other efficient procedures into the exploration of the neighborhood system. The adopted improvement procedures are the iterated greedy algorithm, the path relinking technique, the greedy randomized adaptive search procedure as well as the local search procedures. The proposed improvement procedures are considered to be among the most effective in constructive solutions in the field of optimization scheduling problem. To verify the efficiency of our algorithms, a simulation study is carried out on a set of instances ranging from that of small to that of larger size. The proposed improvement in the algorithms concerns the exploration phase of the neighborhood of the current solution. The present study shows that the water wave algorithm performs good results compared to the migratory bird optimization algorithm.

In future, several lines of research are open to continue the investigation in this kind of problems. The optimization of other criteria

such as energy consumption or minimization of machine loads can be the subject of future potential works. Likewise, a multi-objective optimization by combining criteria like the makespan and the total flow time can present an interesting study with great utility for industrial scheduling decision making.

*Data availability*

The data used the support the findings of this study are available from the corresponding author upon request.

## CRediT authorship contribution statement

**Said Aqil:** Conceptualization, Methodology, Software, Writing - original draft, Visualization, Investigation, Writing - reviewing and editing. **Karam Allali:** Conceptualization, Methodology, Software, Writing - original draft, Visualization, Investigation, Writing - reviewing and editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References


Alekseeva, E., Mezmaz, M., Tuyttens, D., Melab, N., 2017. Parallel multi-core hyper-heuristic grasp to solve permutation flow-shop problem. Concurr. Comput.: Pract. Exper. 29 (9), e3835.

Aqil, S., Allali, K., 2019. On a bi-criteria flow shop scheduling problem under constraints of blocking and sequence dependent setup time. Annals of Operations Research 1–23.

Aqil, S., Allali, K., 2020. Local search metaheuristic for solving hybrid flow shop problem in slabs and beams manufacturing. Expert Syst. Appl. 162, 113716.

Benkalai, I., Rebaine, D., Gagné, C., Baptiste, P., 2017. Improving the migrating birds optimization metaheuristic for the permutation flow shop with sequence-dependent set-up times. Int. J. Prod. Res. 55 (20), 6145–6157.

Chamnanlor, C., Sethanan, K., Gen, M., Chien, C.-F., 2017. Embedding ant system in genetic algorithm for re-entrant hybrid flow shop scheduling problems with time window constraints. J. Intell. Manuf. 28 (8), 1915–1931.

Cui, Z., Gu, X., 2015. An improved discrete artificial bee colony algorithm to minimize the makespan on hybrid flow shop problems. Neurocomputing 148, 248–259.

Davoudpour, H., Ashrafi, M., 2009. Solving multi-objective sdst flexible flow shop using grasp algorithm. Int. J. Adv. Manuf. Technol. 44 (7–8), 737–747.

Duman, E., Uysal, M., Alkaya, A.F., 2012. Migrating birds optimization: A new metaheuristic approach and its performance on quadratic assignment problem. Inform. Sci. 217, 65–77.

Ernst, A., Fung, J., Singh, G., Zinder, Y., 2019. Flexible flow shop with dedicated buffers. Discrete Appl. Math. 261, 148–163.

Fernandez-Viagas, V., Perez-Gonzalez, P., Framinan, J.M., 2019. Efficiency of the solution representations for the hybrid flow shop scheduling problem with makespan objective. Comput. Oper. Res. 109, 77–88.

González-Neira, E.M., Montoya-Torres, J.R., 2017. A grasp meta-heuristic for the hybrid flowshop scheduling problem. J. Decis. Syst. 26 (3), 294–306.

Graham, R.L., Lawler, E.L., Lenstra, J.K., Kan, A.R., 1979. Optimization and approximation in deterministic sequencing and scheduling: a survey. In: Annals of Discrete Mathematics, Vol. 5. Elsevier, pp. 287–326.

Jun, S., Park, J., 2015. A hybrid genetic algorithm for the hybrid flow shop scheduling problem with nighttime work and simultaneous work constraints: A case study from the transformer industry. Expert Syst. Appl. 42 (15–16), 6196–6204.

Khalouli, S., Ghedjati, F., Hamzaoui, A., 2010. A meta-heuristic approach to solve a JIT scheduling problem in hybrid flow shop. Eng. Appl. Artif. Intell. 23 (5), 765–771.

Khare, A., Agrawal, S., 2019. Scheduling hybrid flowshop with sequence-dependent setup times and due windows to minimize total weighted earliness and tardiness. Comput. Ind. Eng. 135, 780–792.

Kianfar, K., Ghomi, S.F., Jadid, A.O., 2012. Study of stochastic sequence-dependent flexible flow shop via developing a dispatching rule and a hybrid GA. Eng. Appl. Artif. Intell. 25 (3), 494–506.

Kurdi, M., 2019. Ant colony system with a novel non-daemonactions procedure for multiprocessor task scheduling in multistage hybrid flow shop. Swarm Evol. Comput. 44, 987–1002.

Lee, H.E., Huh, E.-H., Yoon, Y., Yoon, S.J., Huh, D.-A., Moon, K.W., 2019. Valuation of estimation toxic chemical release inventory method-focusing on paint manufacturing process. Int. J. Environ. Res. Public Health 16 (18), 3260.

Li, K., Chen, J., Fu, H., Jia, Z., Fu, W., 2019. Uniform parallel machine scheduling with fuzzy processing times under resource consumption constraint. Appl. Soft Comput. 105585.

Li, K., Zhang, H.-J., Cheng, B.-Y., Pardalos, P.M., 2018. Uniform parallel machine scheduling problems with fixed machine cost. Optim. Lett. 12 (1), 73–86.

Lin, C.-C., Liu, W.-Y., Chen, Y.-H., 2020. Considering stockers in reentrant hybrid flow shop scheduling with limited buffer capacity. Comput. Ind. Eng. 139, 106154.

Moccellin, J.V., Nagano, M.S., Neto, A.R.P., de Athayde Prata, B., 2018. Heuristic algorithms for scheduling hybrid flow shops with machine blocking and setup times. J. Braz. Soc. Mech. Sci. Eng. 40 (2), 40.

Mollaei, A., Mohammadi, M., Naderi, B., 2019. A bi-objective milp model for blocking hybrid flexible flow shop scheduling problem: robust possibilistic programming approach. Int. J. Manage. Sci. Eng. Manage. 14 (2), 137–146.

Nawaz, M., Enscore, Jr., E.E., Ham, I., 1983. A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. Omega 11 (1), 91–95.

Newton, M., Riahi, V., Su, K., Sattar, A., 2019. Scheduling blocking flowshops with setup times via constraint guided and accelerated local search. Comput. Oper. Res. 109, 64–76.

Osman, I.H., Potts, C., 1989. Simulated annealing for permutation flow-shop scheduling. Omega 17 (6), 551–557.

Öztop, H., Tasgetiren, M.F., Eliiyi, D.T., Pan, Q.-K., 2019. Metaheuristic algorithms for the hybrid flowshop scheduling problem. Comput. Oper. Res..

Pan, Q.-K., Dong, Y., 2014. An improved migrating birds optimisation for a hybrid flowshop scheduling with total flowtime minimisation. Inform. Sci. 277, 643–655.

Pan, Q.-K., Ruiz, R., Alfaro-Fernández, P., 2017. Iterated search methods for earliness and tardiness minimization in hybrid flowshops with due windows. Comput. Oper. Res. 80, 50–60.

Ribas, I., Leisten, R., Framiñan, J.M., 2010. Review and classification of hybrid flow shop scheduling problems from a production system and a solutions procedure perspective. Comput. Oper. Res. 37 (8), 1439–1454.

Ruiz, R., Vázquez-Rodríguez, J.A., 2010. The hybrid flow shop scheduling problem. European J. Oper. Res. 205 (1), 1–18.

Sioud, A., Gagné, C., 2018. Enhanced migrating birds optimization algorithm for the permutation flow shop problem with sequence dependent setup times. European J. Oper. Res. 264 (1), 66–73.

Takano, M., Nagano, M., 2019. Evaluating the performance of constructive heuristics for the blocking flow shop scheduling problem with setup times. Int. J. Ind. Eng. Comput. 10 (1), 37–50.

Valizadeh, S., Fatahi Valilai, O., Houshmand, M., 2019. Flexible flow line scheduling considering machine eligibility in a digital dental laboratory. Int. J. Prod. Res. 1–19.

Vignier, A., Billaut, J.-C., Proust, C., 1999. Les problèmes d'ordonnancement de type «flow-shop» hybride: état de l'art. RAIRO Oper. Res. Rech. Oper. 33 (2), 117–183.

Wang, S., Wu, R., Chu, F., Yu, J., 2019. Variable neighborhood search-based methods for integrated hybrid flow shop scheduling with distribution. Soft Comput. 1–20.

Yu, C., Semeraro, Q., Matta, A., 2018. A genetic algorithm for the hybrid flow shop scheduling with unrelated machines and machine eligibility. Comput. Oper. Res. 100, 211–229.

Zabihzadeh, S.S., Rezaeian, J., 2016. Two meta-heuristic algorithms for flexible flow shop scheduling problem with robotic transportation and release time. Appl. Soft Comput. 40, 319–330.

Zhang, M., 2019. Flexible flow shop scheduling problem with setup times and blocking constraint via genetic algorithm and simulation. In: IOP Conference Series: Materials Science and Engineering, Vol. 637. IOP Publishing, 012007.

Zhang, B., Pan, Q.-k., Gao, L., Zhang, X.-l., Sang, H.-y., Li, J.-q., 2017. An effective modified migrating birds optimization for hybrid flowshop scheduling problem with lot streaming. Appl. Soft Comput. 52, 14–27.

Zhao, F., Liu, H., Zhang, Y., Ma, W., Zhang, C., 2018. A discrete water wave optimization algorithm for no-wait flow shop scheduling problem. Expert Syst. Appl. 91, 347–363.

Zhao, F., Zhang, L., Zhang, Y., Ma, W., Zhang, C., Song, H., 2019. A hybrid discrete water wave optimization algorithm for the no-idle flowshop scheduling problem with total tardiness criterion. Expert Syst. Appl. 113166.

Zheng, Y.-J., 2015. Water wave optimization: a new nature-inspired metaheuristic. Comput. Oper. Res. 55, 1–11.

Zohali, H., Naderi, B., Mohammadi, M., Roshanaei, V., 2019. Reformulation, linearization, and a hybrid iterated local search algorithm for economic lot-sizing and sequencing in hybrid flow shop problems. Comput. Oper. Res. 104, 127–138.